

ÉCOLE NATIONALE D'INGÉNIEURS DE BREST



— Cours d'Informatique S6 —

Conduite de Projets Objet (UML)

CÉDRIC BUCHE

buche@enib.fr

version du 28 août 2019

Conduite de Projets Objet

— UML —

Cédric Buche

École Nationale d'Ingénieurs de Brest (ENIB)

28 août 2019



Table des matières

— Cours —	3
1 Diagramme d'états-transitions	3
2 Diagramme d'activités	33
3 Génération de code	49
— Labo —	88
4 En attendant ...	89
5 Le modèle états-transitions : concepts de base	94
6 Le modèle états-transitions : concepts avancés	97
7 Le modèle des activités : processus métier	103
8 Le modèle des activités : du modèle au programme	106
— Solution Labo —	111
9 Le modèle états-transitions	111
10 Le modèle d'activités	117

Ces notes de cours accompagnent les enseignements d'informatique du 6^{ème} semestre (S6) de l'École Nationale d'Ingénieurs de Brest (ENIB : www.enib.fr). Leur lecture ne dispense en aucun cas d'une présence attentive aux cours ni d'une participation active aux travaux dirigés.

Une partie de ce document est librement inspiré de l'ouvrage de Laurent Audibert. Une partie des supports ont été rédigés par Pierre Chevaillier.

Cours

1 Diagramme d'états-transitions

Sommaire

1.1	Introduction	5
1.1.1	Définitions — rôles	5
1.1.2	Représentation graphique	6
1.1.3	Exemple : télérupteur dans une maison	6
1.1.4	Interprétations du modèle	6
1.2	Bases	7
1.2.1	Concepts de base	7
1.2.2	Notion d'état	7
1.2.3	État initial et état final	8
1.2.4	Transition	9
1.2.5	Événement	11
1.2.6	Transition d'achèvement	12
1.2.7	Condition de garde d'une transition	13
1.2.8	Effet associé à une transition	13
1.2.9	Exemples	14
1.2.10	Machine à états	15
1.2.11	Exercices	16
1.3	Concepts avancés	18
1.3.1	Activités et transitions internes	18
1.3.2	Alternatives	19
1.3.3	État composite	21
1.3.4	Parallélisme	23
1.3.5	Transitions complexes	24
1.3.6	Hierarchie	25
1.3.7	Exercices	28
1.4	Mise en œuvre	31
1.4.1	Un modèle d'états-transitions pour quelles classes ?	31

1.4.2	Comment construire un diagramme d'états-transitions?	32
1.4.3	Quand utiliser les diagrammes d'états-transitions?	32

1.1 Introduction

1.1.1 Définitions — rôles

Les diagrammes d'états-transitions UML proposent une *vue dynamique* du comportement d'un classifieur (évolutions possibles des états).

Objectifs Il s'agit de décrire le comportement interne d'un classifieur :

- ▷ Ensemble des **états** possibles (pertinents)
- ▷ Possibilités de **changements d'état** sur occurrence d'événement

Origine Les diagrammes d'états-transitions UML sont fondés sur les *Statecharts* (David HARREL, 1987). Plus précisément, les diagrammes d'états-transitions UML utilisent des machines à états.

Machine à états Le comportement ne dépend pas uniquement des entrées du système, mais aussi de son « histoire », son état interne

Type de modèle Les diagrammes d'états-transitions UML permettent de modéliser un système discret :

- ▷ comportement caractérisable (= observable) par une succession d'états
- ▷ avec changement d'état discontinu (= discret)

Utilisation pratique Les diagrammes d'états-transitions UML permettent classiquement de préciser le comportement interne d'une instance de classe, mais ils peuvent être utilisés aussi pour d'autres éléments comme une méthode ou un cas d'utilisation. Les diagrammes d'états-transitions est le seul diagramme UML à offrir une vision complète de l'ensemble des comportements de l'élément auquel il est attaché. En effet, un diagramme d'interaction (séquence / communication) n'offre qu'une vue partielle correspondant à un cas d'utilisation.

1.1.2 Représentation graphique

Un automate à états finis est graphiquement représenté en UML par un graphe comportant des états, matérialisés par des rectangles aux coins arrondis, et des transitions, matérialisées par des arcs orientés reliant les états entre eux.

1.1.3 Exemple : télérupteur dans une maison

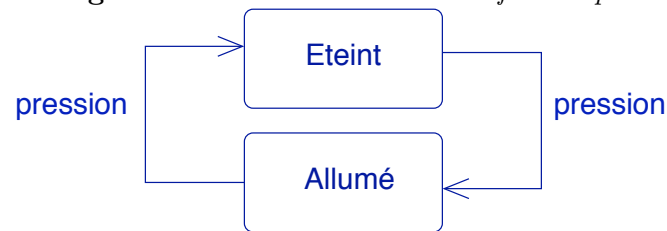
Lorsque l'on appuie sur un bouton poussoir, la réaction de l'éclairage associé dépend de son état courant (donc de son historique) :

- ▷ si la lumière est allumée, elle s'éteint,
- ▷ si elle est éteinte, elle s'allume.

Les éléments du comportement du système :

- ▷ 2 états : *Allumé* et *Éteint*
- ▷ 2 transitions : $Allumé \mapsto Éteint$ et $Éteint \mapsto Allumé$
- ▷ 1 événement : *pression* sur un bouton poussoir

Figure 1.1. *Un automate d'états fini simple*



1.1.4 Interprétations du modèle

Deux interprétations sont possibles :

1. Machine à état comportementale.

Il s'agit de spécifier le comportement d'un classifieur, instance d'une classe, cas d'utilisation, collaboration, méthode : **ce que fait un classifieur** quand on le sollicite. Graphiquement, il faut indiquer **stm** {behavior}.

2. Machine à état de protocole.

Il s'agit de spécifier les séquencements « légaux » de sollicitations : **comment utiliser un classifieur**, sans effet sur une instance du classifieur. Graphiquement, il faut indiquer **stm** {protocol}.

1.2 Bases

Un diagramme d'états-transitions ne peut être associé qu'à un seul classifieur. Tous les automates à états-finis des diagrammes d'états-transitions d'un système s'exécutent concurremment et peuvent changer d'état de façon indépendante.

1.2.1 Concepts de base

Le diagramme d'états-transitions UML conserve les principes généraux des modèles états – transition : État, Transition, Événement.

Le diagramme d'états-transitions UML ajoute des spécificités : Signal, Transition gardée, Effet (machine à état comportementale).

1.2.2 Notion d'état

État simple (sens strict) :

- ▷ potentiellement *observable au cours de la vie* d'un classifieur
- ▷ *durée d'activation non nulle*
- ▷ défini par un invariant
 - ◇ les valeurs d'un ensemble de propriétés du classifieur
 - ◇ une situation d'attente
 - ◇ l'exécution d'un comportement

Pseudo-état :

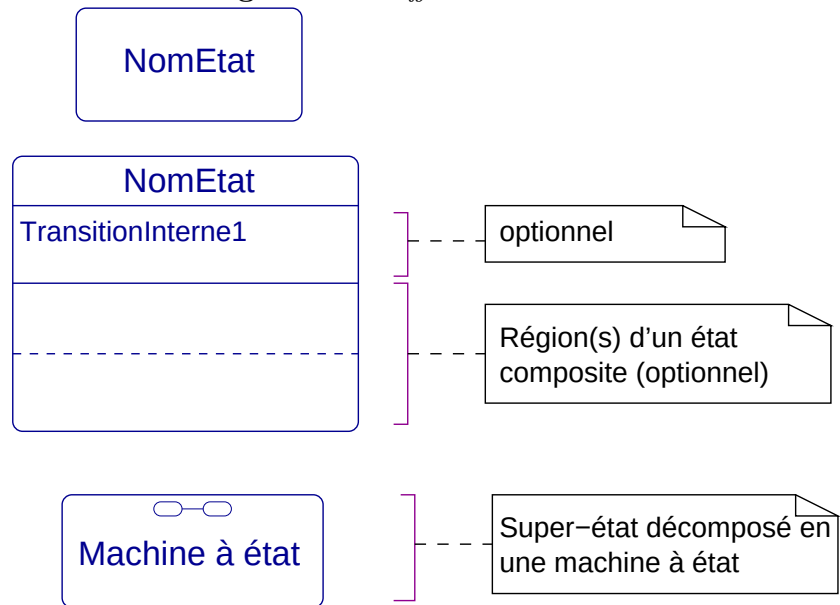
- ▷ non observable, généralement de durée nulle
- ▷ rôle : contrôle des activations des états

État composite

- ▷ peut contenir (envelopper) des sous-états

Notations

Figure 1.2. *Différentes notations*



1.2.3 État initial et état final

Définitions Les états initiaux et finaux sont des pseudo-états caractérisent l'activation (initiale ou terminale) d'une *région* d'un état englobant

État initial (durée nulle) : Lorsqu'un objet est créé, il entre dans l'état initial. L'état initial indique l'activation initiale de la région englobante en ciblant le premier état actif par défaut.

Attention, il existe des restrictions sur la Transition d'un état initial :

- ▷ non déclenchée par un événement,

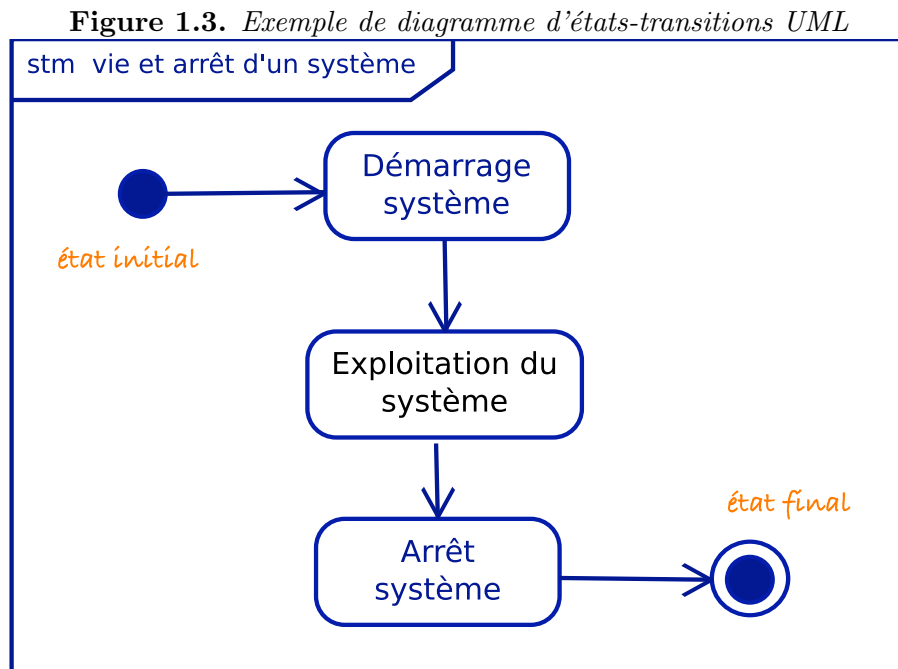
- ▷ éventuellement gardée (sous réserve...),
- ▷ éventuellement associée à un *effet*

Un état initial est représenté par un rond plein.

État final L'état final indique que l'activation de la région englobante est terminée. Aucun autre état n'est alors activable.

Un état final est représenté par un rond plein entouré d'un cercle.

Exemple

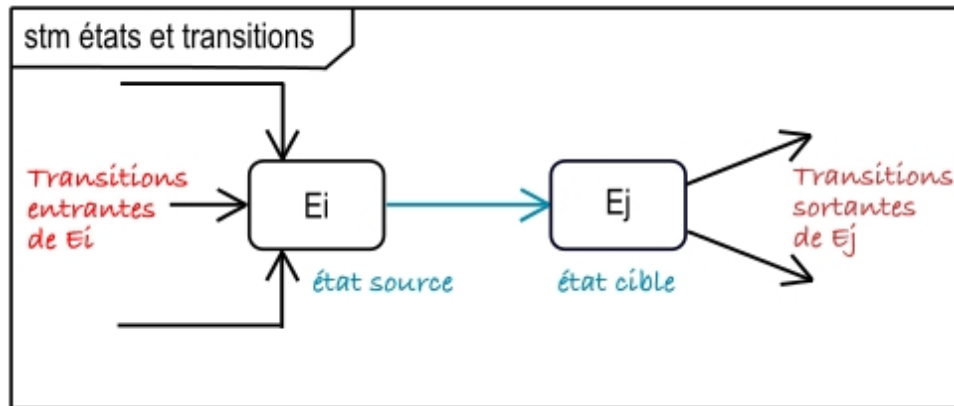


1.2.4 Transition

Définition Une transition indique un changement possible d'état du classier. Elle définit la réponse d'un objet à l'occurrence d'un événement. Une transition relie un **état source** à un **état**

cible.

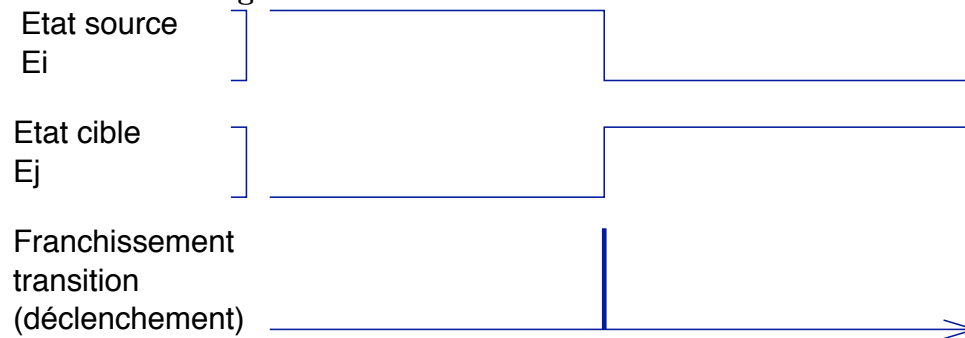
Figure 1.4. Principe de transition



Attention : ce diagramme n'est pas syntaxiquement correct. En effet, une transition relie nécessairement 2 états. Cette figure correspond à un extrait du diagramme.

Déclenchement Le changement d'état est instantané (= atomique). Le franchissement (= déclenchement) d'une transition a lieu sur **occurrence d'un événement**. Il peut aussi dépendre d'une **condition de garde**. Le franchissement peut avoir un **effet** sur le classif = réalisation d'une action.

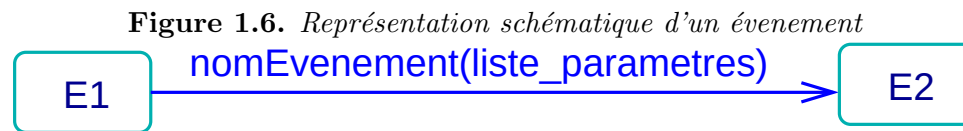
Figure 1.5. Franchissement d'une transition



1.2.5 Événement

Définition Un événement est quelque chose qui se produit à un instant donné lors de l'exécution d'un système. Un événement peut véhiculer une information (paramètres) entre un objet émetteur et un objet récepteur. Un événement est associé à une transition (ou plusieurs) l'**occurrence d'un événement** est caractérisée par :

- ▷ une instance d'un type d'événement
- ▷ est instantané (durée nulle)
- ▷ peut provoquer le déclenchement d'une transition (éventuellement plusieurs)



Remarques :

- ▷ une transition peut ne pas avoir d'événement associé ;
- ▷ quand un événement est associé à une transition, on dit aussi qu'elle est synchronisée sur l'événement ;
- ▷ on ne peut pas associer plus d'un événement à une transition ;
- ▷ un même type d'événement peut être associé à plusieurs transitions ;
- ▷ une transition peut à la fois avoir une garde et être synchronisée sur un événement.

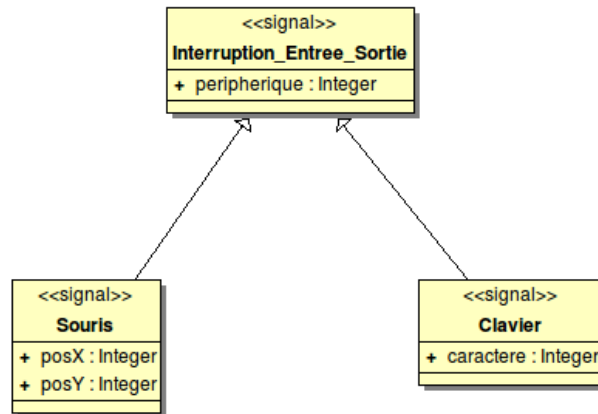
Types prédéfinis

- ▷ `call event : nomOpe(parameter: type, ...)`
 - ◇ réception d'un appel d'invocation d'une opération
 - ◇ le déclenchement de la transition provoque l'exécution de l'opération
 - ◇ l'émetteur retrouve le contrôle lorsque l'opération est exécutée
- ▷ `change event : when(<expression>)`
 - ◇ événement occurrent quand <expression> devient vraie

- ◊ pas de paramètre
- ▷ `signal event : nomSignal(parameter: type, ...)`
 - ◊ communication asynchrone entre objets
 - ◊ (voir la suite)
- ▷ `time event : after(<time>)`
 - ◊ `<time>` : expression temporelle spécifiant un instant ou durée

Événement de type Signal Un signal est un stéréotype de classifier. Un `<<signal>>` peut avoir des attributs et éventuellement des opérations (p. ex. pour sa création). Un `<<signal>>` peut être défini comme une spécialisation d'un autre; la réception d'un signal event provoque le déclenchement d'une transition étiquetée par une de ses généralisations. Il sert à la **communication asynchrone explicite** entre objets. Il est produit par une action `send` et peut être destiné à un ou plusieurs objets (explicites).

Figure 1.7. Exemple de déclaration de signaux



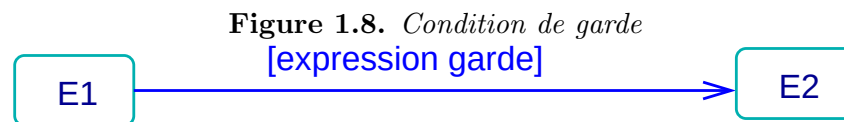
1.2.6 Transition d'achèvement

Une transition d'achèvement est une transition dépourvue d'événement déclencheur explicite. Elle se déclenche à la fin de l'exécution de l'activité de l'état source (y compris les états

imbriqués). Elle peut avoir une condition de garde. Celle-ci est évaluée quand l'activité de l'état source s'achève ; elle ne l'est plus ensuite.

1.2.7 Condition de garde d'une transition

La condition de garde d'une transition est une **expression booléenne** évaluée dans le contexte de l'objet (classifieur) détenteur de la machine à état. C'est une expression logique sur les attributs de l'objet, ainsi que sur les paramètres de l'événement déclencheur. Elle n'est *évaluée que si la transition est éligible* à son déclenchement. En pratique, elle peut correspondre à une opération ayant un paramètre de retour de type `boolean` ; l'opération doit être une requête (`{query}`).



La garde est évaluée au moment du franchissement de la transition. Son évaluation ne doit pas avoir d'effet de bord, c'est à dire qu'elle ne doit pas modifier l'état du système, ni produire d'événement.

Attention : il ne faut pas confondre une condition de garde et un événement de type `change event`. Sauriez-vous expliquer la différence ? Dans la pratique, une transition d'achèvement gardée, dans le cas où il n'y a pas d'activité associée à l'état source, ne pose pas de problème : la transition est franchie quand la condition de garde est vraie (cette dernière peut devenir vraie avant ou pendant que le classifieur est dans l'état source, cela ne change rien). Dans le cas où il y a une activité associée à l'état source, il est prudent d'indiquer ce qu'il faut faire si cette activité prend fin alors que la condition de garde n'est pas vraie. Dans le cas contraire il y aurait un blocage potentiel pas très explicite sur le diagramme.

1.2.8 Effet associé à une transition

L'effet associé à une transition est un **comportement** exécuté lors du franchissement d'une transition. Il peut modifier l'objet détenteur de la machine à état. Il peut utiliser les paramètres de l'événement déclencheur (événement en cours).

Il est lié à une *sémantique run-to-completion* : aucun nouvel événement n'est traité pendant son exécution ; plusieurs actions peuvent être associées à la même transition. En pratique il faut se limiter à des actions « courtes » (voir transitions internes pour les actions « étendues »).

Figure 1.9. *Représentation schématique d'un effet*



Le comportement associé à une transition peut être :

- ▷ une opération primitive comme une instruction d'assignation (modification d'un attribut ou d'un lien de l'objet auquel correspond la machine à état ;
- ▷ l'envoi d'un signal ;
- ▷ l'appel d'une opération ;
- ▷ une liste d'actions (expression des actions séparées par des virgules
- ▷ ...

1.2.9 Exemples

Figure 1.10. *Exemples de transition*
evenement(params)[cond]/active

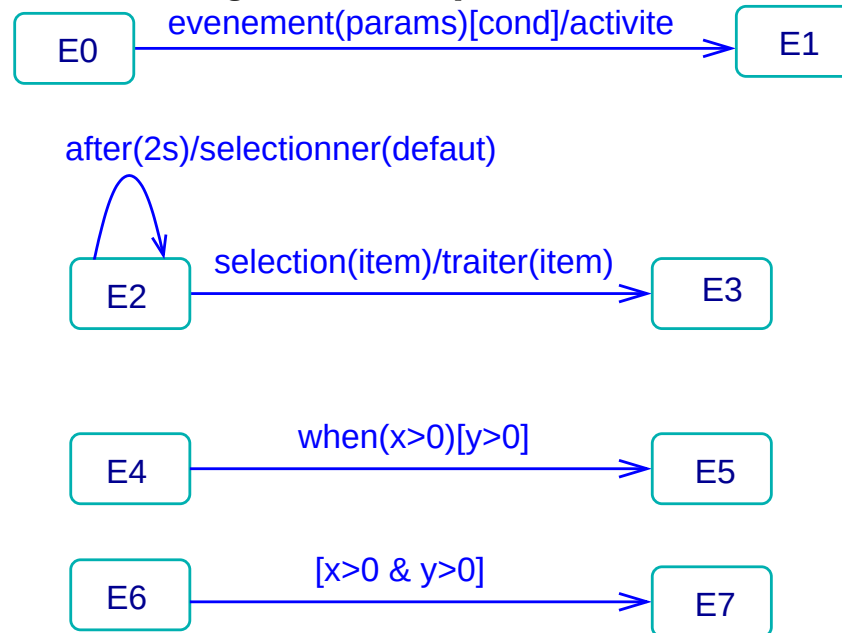
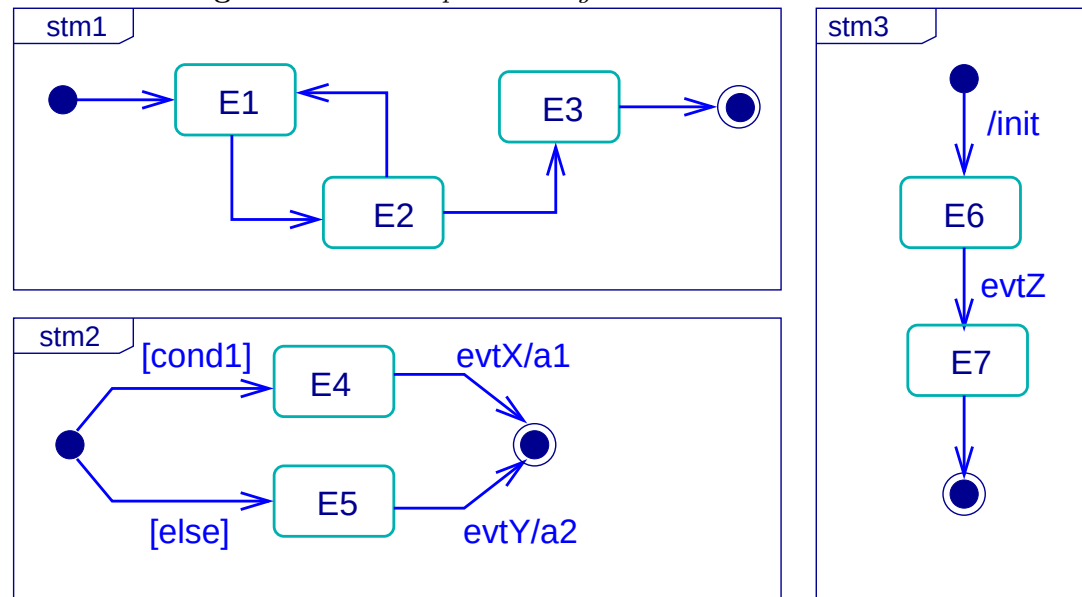


Figure 1.11. Exemples de diagrammes d'états-transitions



Note : il a des restrictions sur l'expression d'une transition ayant comme état source un état initial. Il n'y en a pas dans le cas d'une transition ayant pour cible un état final.

1.2.10 Machine à états

Sémantique des machines à état La spécification des séquences d'activation des états permet de définir comment traiter les occurrences d'événements ? Quand ? Dans quel ordre ? Quelle(s) transition(s) déclencher ? 0 ou 1 dans le cas simple ; éventuellement plusieurs dans le cas général. Quels sont les états actifs ? 1 seul dans le cas simple (plusieurs dans le cas des états composites ou hiérarchiques).

La sémantique des machines à état UML diffère des *statemachines* de HAREL ; elle a fortement évolué d'une version d'UML à une autre.

Occurrences d'événements

- ▷ Modèle **asynchrone** : deux événements ne peuvent se produire simultanément et ils sont indépendants
- ▷ Modèle **discret** : on ne traite qu'un événement à la fois
- ▷ Les événements produits sont placés dans un **pool d'événements**
 - ◊ le pool peut être vide ou contenir plusieurs événements
 - ◊ conceptuellement, l'événement traité est choisi arbitrairement
une **sémantique opérationnelle** doit spécifier cet ordre aléatoire ou *file* (=PAPS)
- ▷ **run-to-completion** : on répercute toutes les conséquences de l'occurrence d'un événement avant d'en traiter un autre

Transitions déclenchées / traitement d'un événement

1. Toutes les transitions étiquetées par cet événement (ou par un événement plus général) et dont l'état source est actif sont **éligibles** (= potentiellement déclenchables)
2. Qu'il y ait ou non une transition éligible, l'événement est consommé (retiré du *pool*)
3. Les gardes associées aux transitions éligibles sont évaluées (l'ordre est indifférent) ;
seules les transitions éligibles dont les conditions de garde sont satisfaites sont déclenchées
4. Les effets associés aux transitions éligibles sont exécutés
5. L'état cible est activé

1.2.11 Exercices

Exo1 On considère la machine à état comportementale suivante d'un classifieur CC.

1. À sa création, le classifieur passe dans l'état E1 après avoir exécuté l'action `init()`.
2. Étant en E1, il passe en E2 sur occurrence d'un événement `evtX` en exécutant l'action `act1`.
3. Une occurrence de `evtY` en E2 rend le classifieur inactif.
4. Étant en E1, si la condition `cond` est vraie, et quand l'événement `evtY` se produit, le classifieur passe dans l'état E3 en exécutant `act2`.

5. Étant en E3, evtY le refait passer en E1.

Représentez ce comportement sous forme d'un diagramme états-transitions UML.

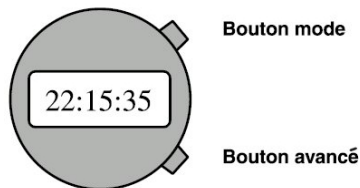
Réveil matin Considérons un réveil-matin simplifié :

- ▷ on peut mettre l'alarme “on” ou “off”,
- ▷ quand l'heure courante devient égale à l'heure d'alarme, le réveil sonne sans s'arrêter,
- ▷ on peut interrompre la sonnerie.

Question 1 – Dessinez le diagramme d'états-transitions correspondant

Question 2 – Complétez le diagramme pour prendre en compte le fait que la sonnerie s'arrête d'elle-même au bout d'un certain temps

Montre à cadran Considérons une montre à cadran numérique simplifiée :



1. Le mode courant est le mode “*affichage*”
2. Quand on appuie une fois sur le bouton mode, la montre passe en “modification heure”. Chaque pression sur le bouton avance incrémente l'heure d'une unité.
3. Quand on appuie une nouvelle fois sur le bouton mode, la montre passe en “modification minute”. Chaque pression sur le bouton avance incrémente les minutes d'une unité.
4. Quand on appuie une nouvelle fois sur le bouton mode, la montre repasse en mode “Affichage”.

Question 1 – Dessinez le diagramme d'états-transitions correspondant
--

Question 2 – Ajoutez le comportement suivant : quand on appuie sur le bouton avance plus de deux secondes, les heures (ou les minutes) s'incrémentent rapidement jusqu'à ce qu'il se produise un relâchement dans la pression du bouton. Envisagez plusieurs solutions possibles
--

1.3 Concepts avancés

1.3.1 Activités et transitions internes

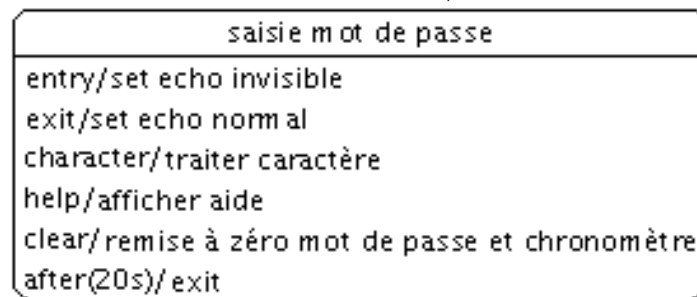
Activités « internes » Les Activités « internes » permettent de préciser ce qu'il se passe au sein d'un état actif. Elles offrent une meilleure encapsulation de la modélisation des comportements en distinguant ce qui dépend de l'environnement (événements – transitions) et de l'état intrinsèque d'un classifier :

- ▷ **do / activity** : activité effectuée quand l'état est actif
elle se termine d'elle-même ou quand l'état n'est plus actif
- ▷ **entry / activity** : activité exécutée lorsque l'état devient actif
elle est exécutée après les activités des transitions d'entrée et avant les activités **do** et **exit**
- ▷ **exit / activity** : activité exécutée lorsque l'état devient inactif
- ▷ **include / activity** : permet d'invoquer un sous-diagramme d'états-transitions

Les activités **entry** servent souvent à effectuer la configuration nécessaire dans un état. Comme il n'est pas possible de l'éluder, toute action interne à l'état peut supposer que la configuration est effectuée indépendamment de la manière dont on entre dans l'état. De manière analogue, une activité **exit** est une occasion de procéder à un nettoyage.

Transitions « internes » Une transition « interne » est une transition attachée à un état (qui doit être actif pour qu'elle soit déclenchée). Elle ne provoque pas de changement d'état donc pas de réalisation des activités internes **entry** ni **exit**. Notation : expression d'une transition mais pas d'arc : `nomEVT(liste param)[garde]/activites`.

Figure 1.12. *Exemple d'activités/transitions internes*

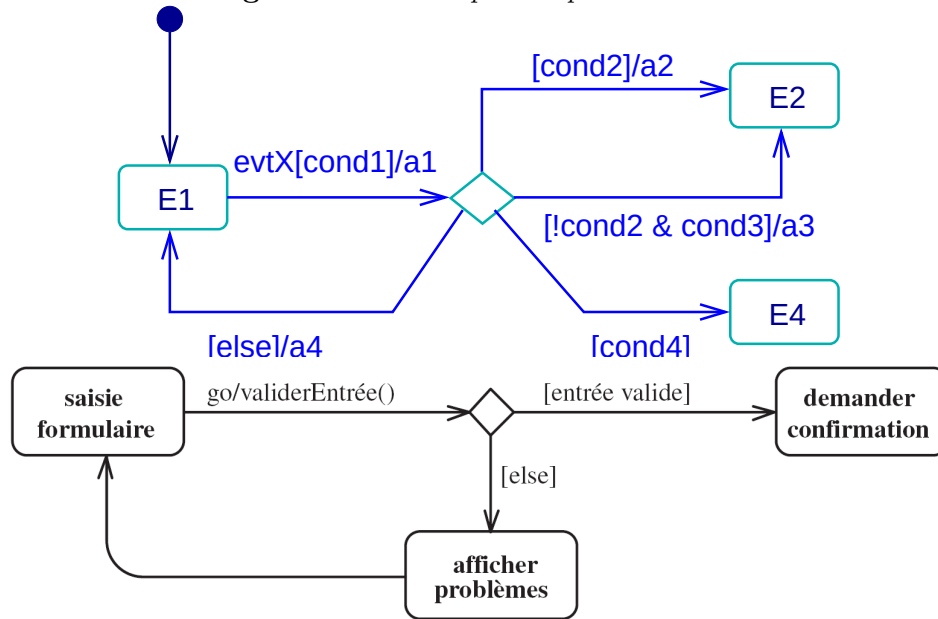


1.3.2 Alternatives

Point de choix et point de jonction Les points de choix et de jonction sont une notation mettant en évidence les alternatives dans les modèles États–Transitions. Ceux sont des Pseudo-état correspondant à une **alternative** liée à l'évaluation des conditions de garde. Dans un modèle bien formé, exactement une condition de garde des transitions de sortie d'un point de choix doit être valide. Les transitions ayant comme état cible ou état source un point de choix (de jonction) sont appelées *segments de transition*. **Différence** : moment auquel les gardes des segments de transition en aval du point de choix sont évaluées. Cette notation renforce la lisibilité des diagrammes et elle est donc très utilisée ; conceptuellement, elle n'est pas nécessaire. Ces points de choix sont aussi appelés *vertex* de choix.

Point de choix (= de décision) Un pseudo-état « point de choix » possède exactement un segment de transition entrant et au moins deux segments sortants. Les gardes situées sur les segments APRÈS le point de décision sont évaluées au moment où ce dernier est atteint. Il est possible d'utiliser une garde particulière, notée `[else]`, sur un des segments en sortie d'un point de choix. Ce segment n'est franchissable que si les gardes des autres segments sont toutes fausses. L'utilisation d'une clause `[else]` est recommandée après un point de décision car elle garantit un modèle bien formé.

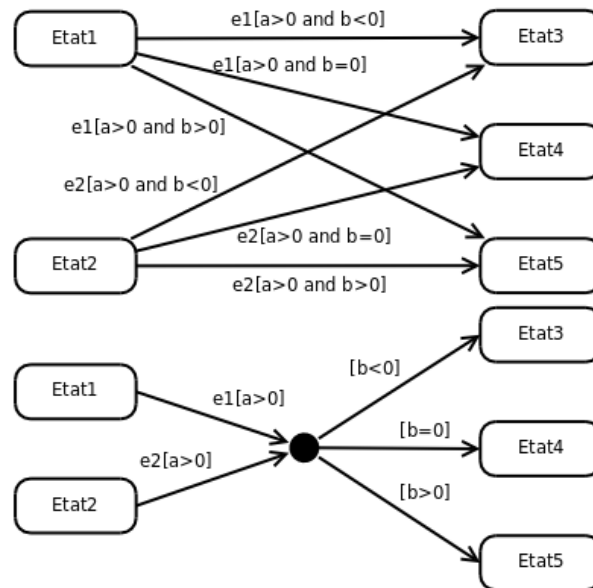
Figure 1.13. *Exemples de point de choix*



Point de jonction Un pseudo-état « Un point de jonction » peut avoir plusieurs segments de transition entrants et plusieurs segments de transition sortants. Seuls les segments entrants peuvent être synchronisés sur des événements. Il ne peut pas avoir d'activité interne, ni des transitions sortantes dotées de déclencheurs d'événements. L'évaluation des gardes a lieu au moment du déclenchement du segment entrant.

Dans la figure suivante, en haut, un diagramme sans point de jonction ; en bas, son équivalent utilisant un point de jonction.

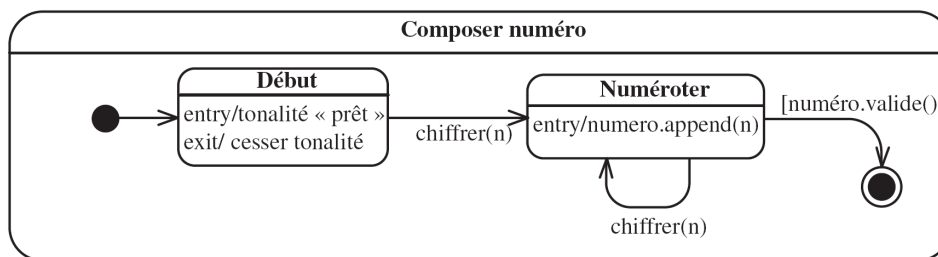
Figure 1.14. *Equivalence point de jonction*



1.3.3 État composite

Présentation Un état simple ne possède pas de sous-structure mais uniquement, le cas échéant, un jeu d'activités et/ou de transitions internes. Un **état composite** est un état décomposé en *régions* contenant chacune un ou plusieurs sous-états. Un état composite peut aussi avoir des activités et des transitions internes. Dans ce cas, il faut faire attention à l'ordre dans lequel les effets associés sont exécutés lors du franchissement d'une transition.

Figure 1.15. Exemple de l'état composite "Composer numéro"



Notation abrégée L'utilisation d'états composites permet de développer une spécification par raffinement. Il n'est pas nécessaire de représenter les sous-états à chaque utilisation de l'état englobant. Une notation abrégée permet d'indiquer qu'un état est composite et que sa définition est donnée sur un autre diagramme.

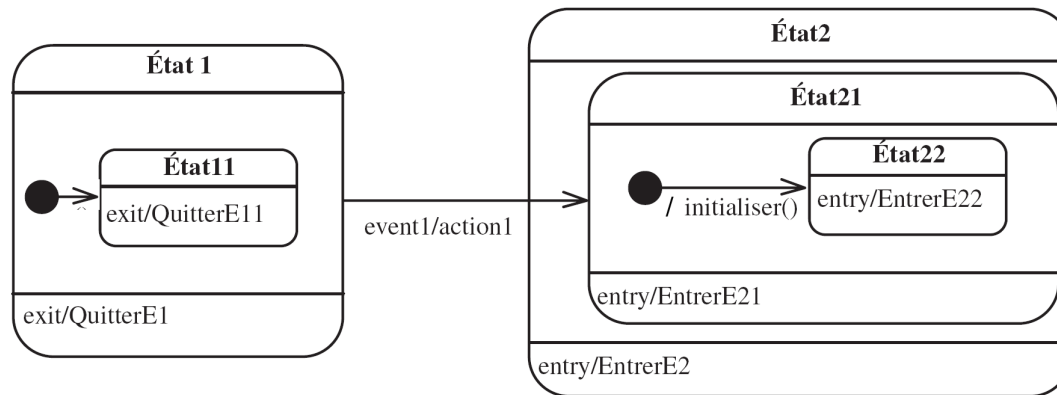
Figure 1.16. *Notation abrégée d'un état composite*



Transition Une transition peut avoir pour cible la frontière d'un état composite. Elle est équivalente à une transition ayant pour *cible l'état initial* de l'état composite cible. Une transition peut avoir pour *source* la frontière d'un état composite. Elle est équivalente à une la transition *s'applique à tout sous-état* de l'état composite source. Relation est transitive : la transition est franchissable depuis tout état imbriqué, quelque soit sa profondeur.

Si une transition ayant pour source la frontière d'un état composite ne porte **pas de déclencheur explicite** (i.e. s'il s'agit d'une transition d'achèvement), elle est franchissable quand l'état final de l'état composite est atteint. Une transition peut avoir comme *source* et *cible* des états de différents niveaux d'imbrication : elle traverse donc les frontières des états composites.

Figure 1.17. *Exemple de transition entre états composites*



Depuis l'état État 1, la réception de l'événement *event1* produit la séquence d'activités : QuitterE11, QuitterE1, action1, EntrerE2, EntrerE21, initialiser(), EntrerE22, et place le système dans l'état État22.

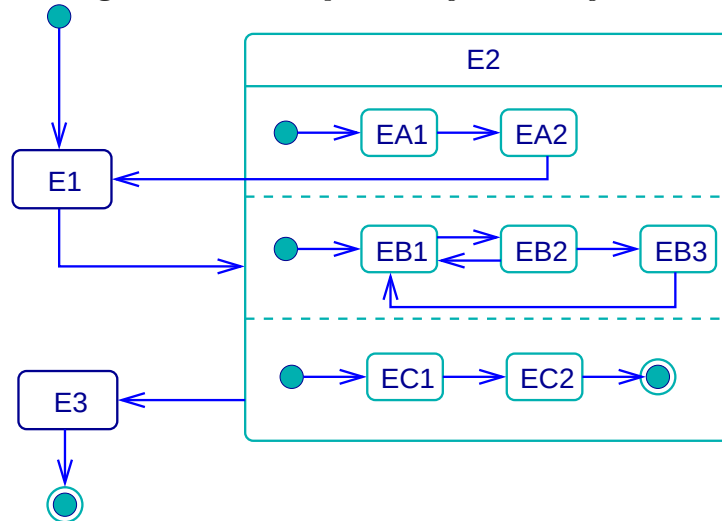
1.3.4 Parallélisme

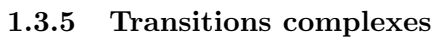
Définitions Le parallélisme (= concurrence) montre les évolutions indépendantes de différents éléments. La dynamique des états est **indépendante** (= *orthogonale*). On modélise un état composite avec régions orthogonales

Région Une région contient un ensemble d'états et de transitions

- ▷ 1 transition peut avoir un état source ou cible « traversant » l'état composite
- ▷ par déf., aucune transition entre états de régions orthogonales
- ▷ 1 et 1 seul état actif par région (si état composite englobant actif)

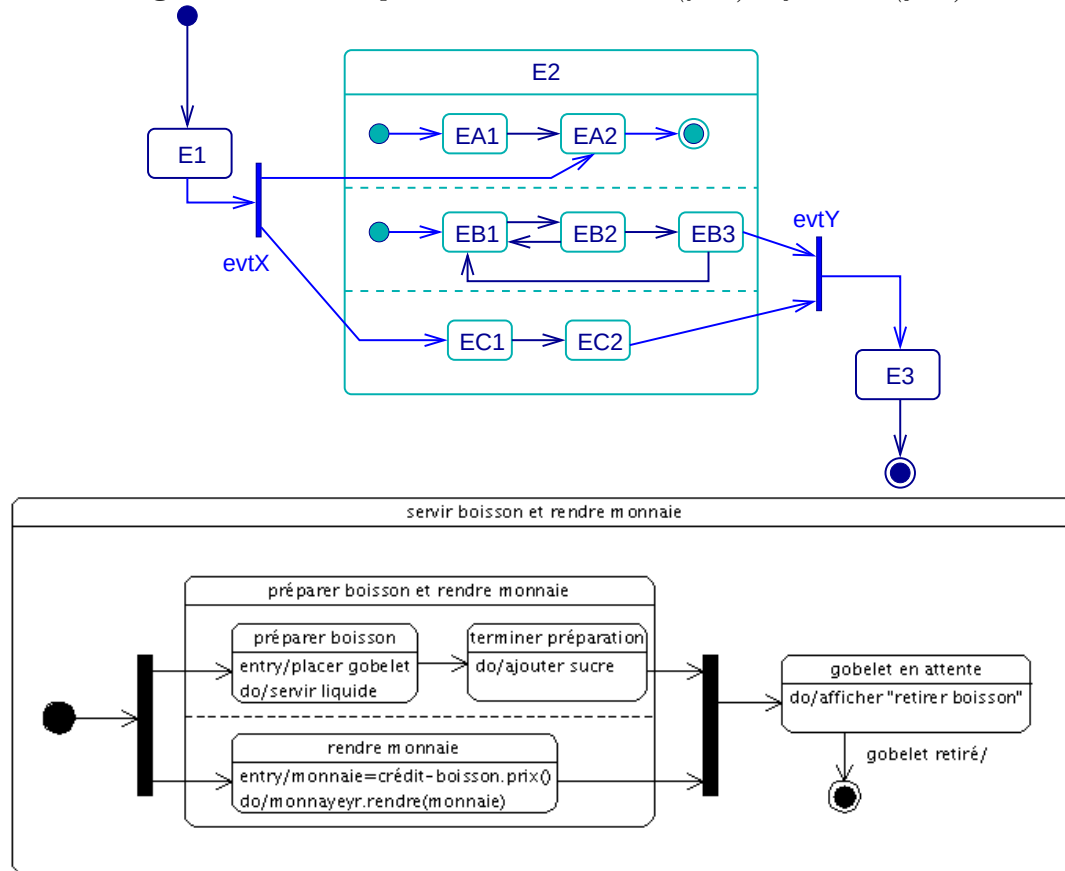
Figure 1.18. Exemples de régions orthogonales





24

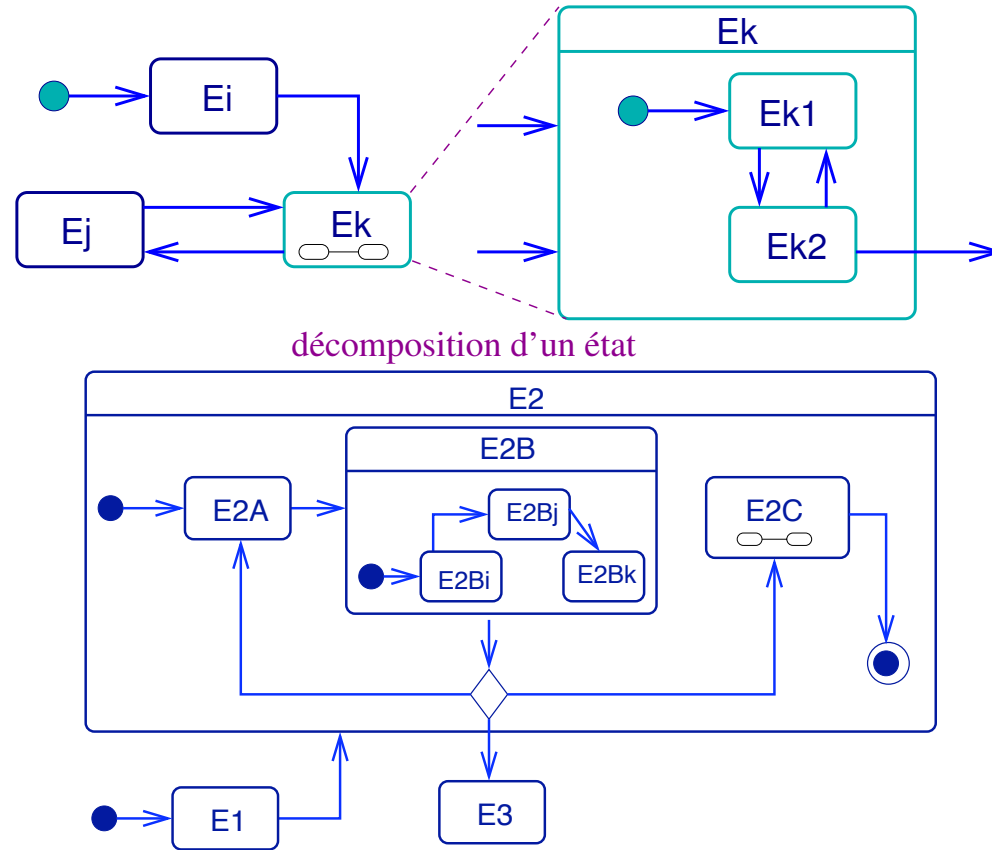
Figure 1.19. Exemples de débranchement (fork) – jointure (join)



1.3.6 Hiérarchie

Le principe de hiérarchie est la décomposition d'un état en une région, elle-même éventuellement décomposée en une région. Les « sous-régions » sont donc non orthogonales : 1 seul état actif dans la hiérarchie. Objectif : **structuration** et **raffinement** de la modélisation.

Figure 1.20. Décomposition d'un état



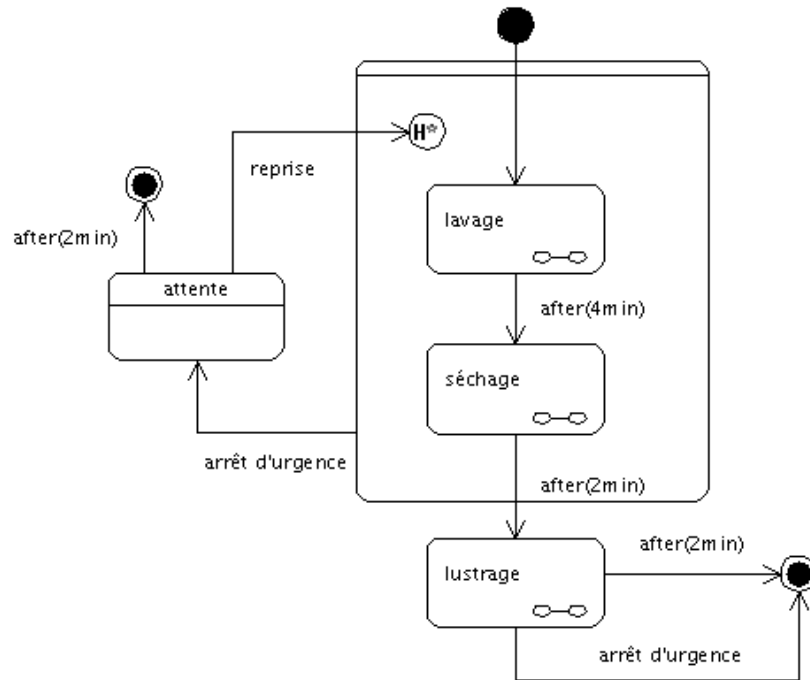
Historique Un pseudo-état *historique* peut être associé à une région : quand elle est activée, c'est le dernier état actif qui est à nouveau activé, et non l'état initial.

Un état historique, également qualifié d'état historique plat, est un pseudo-état qui mémorise le dernier sous-état actif d'un état composite. Graphiquement, il est représenté par un cercle contenant un H. Une transition ayant pour cible l'état historique est équivalente à une transition qui a pour cible le dernier état visité de l'état englobant. Un état historique peut avoir une transition sortante non étiquetée indiquant l'état à exécuter si la région n'a pas encore été

visitée.

Il est également possible de définir un état historique profond représenté graphiquement par un cercle contenant le symbole H^* . Cet état historique profond permet d'atteindre le dernier état visité dans la région, quelque soit son niveau d'imbrication, alors que l'état historique plat limite l'accès aux états de son niveau d'imbrication.

Figure 1.21. *Exemple d'historique*



Les états de lavage, séchage et lustrage sont des états composites définis sur trois autres diagrammes d'états-transitions non représentés ici. En phase de lavage ou de séchage, le client peut appuyer sur le bouton d'arrêt d'urgence. S'il appuie sur ce bouton, la machine se met en attente. Il a alors deux minutes pour reprendre le lavage ou le lustrage, exactement où le programme a été interrompue, c'est à dire au niveau du dernier sous-état actif des états de lavage ou de lustrage (état historique profond). Si l'état avait été un état historique plat, c'est toute la séquence de lavage ou de lustrage qui aurait recommencée. En phase de lustrage, le client peut aussi interrompre la machine. Mais dans ce cas, la machine s'arrêtera définitivement.

1.3.7 Exercices

Exo1 On considère la machine à état comportementale suivante d'un classifieur CC dont le cycle de vie se décompose en trois grandes phases : **Initialisation**, **Exploitation** et **Terminaison**.

1. À sa création, le classifieur est dans l'état **Initialisation**.
2. Ensuite, sur occurrence de **ex**, deux comportements sont activés dont les états initiaux sont respectivement **E1** et **E2**. Ces deux comportements indépendants correspondent à la phase d'**Exploitation**.
3. De l'état **E1**, sur occurrence de **ey** on passe en exploitation (**E3**), état dans lequel on exécute l'action **aa()** ; le comportement est ensuite inactif.
4. L'état **E2** est un état composite (on ne décrit pas ici ce qui s'y passe) dont on sort si la condition **fini** est vraie ; le comportement est ensuite inactif.
5. Quand les deux comportements de la phase d'**Exploitation** sont terminés, l'objet passe dans l'état **Terminaison**, ce qui termine son cycle de vie.

Représentez ce comportement sous forme d'un diagramme états-transitions UML.

Régions concurrentes Reprenons l'exemple de montre à cadran numérique, et ajoutons maintenant à cette dernière deux autres boutons :



- ▷ un bouton éclairage ; en le pressant, on éclaire le cadran de la montre, jusqu'à ce qu'on le relâche ;
- ▷ un bouton alarme, qui ajoute à la montre digitale une fonctionnalité classique d'alarme, comme cela a été décrit lors du premier exercice de ce chapitre (réveille-matin).

Question – Dessinez le diagramme d'états complet incluant tous les comportements de la montre.

Grapher On s'intéresse ici à la modélisation du comportement du grapher quand l'utilisateur interagit avec cet objet graphique au clavier ou avec la souris. On suppose que le système de gestion de l'interface utilisateur produit des signaux du type `KB(n: string)` lorsque l'utilisateur presse une touche nommée ‘‘n’’ et `Mouse` s'il utilise la souris et que le grapher peut y réagir.

1. Au départ le grapher est vide : pas de surface affichée.
2. Si l'on fait appel à l'opération `display(in s: Surface)`, alors cela a pour effet d'exécuter l'action `draw(in s: Surface)`. Une surface est maintenant affichée.
3. Si une surface est affichée, presser la touche “+” déclenche `zoomIn()` et la touche “-” `zoomOut()`. La touche “space” supprime la surface du grapher.
4. Toujours quand une surface est affichée, presser la touche “z” fait passer le grapher dans un mode où l'on affiche un plan de coupe sur l'axe Oz. Lorsqu'on active ce mode, le plan de coupe est affiché, ainsi que sa position selon Oz. Si l'on bouge la souris, cela déplace le plan (`movePlan()`); si l'on appuie sur la touche “space”, cela masque la position; une nouvelle pression sur la touche “space” rend la position de nouveau visible. Dans tous les cas, presser la touche “z” fait sortir du mode plan de coupe.

Représentez ce comportement sous forme d'un modèle état-transition d'UML.

Photos : Prise d'une photo On s'intéresse ici à ce qui se passe juste avant, pendant et après la prise d'une photo avec un appareil numérique. Cela concerne donc la manipulation du déclencheur de l'appareil. Ce bouton est soit relâché, soit enfoncé à mi-course, soit enfoncé à fond.

1. Au départ, le déclencheur est relâché.
2. Une fois que l'on a enfoncé le déclencheur à mi-course, l'appareil effectue la mise au point.
3. À partir de là, si l'on enfonce le déclencheur à fond, cela déclenche la prise de la photo.
4. Notons que si l'on enfonce directement le déclencheur à fond, l'appareil prend une photo, sans changer de réglage.

5. Quand la prise de vue est achevée, l'appareil enregistre la photo sur le support de stockage (cette opération peut prendre un certain temps). La prise de vue est ensuite considérée comme terminée.

Modélisez ce comportement sous forme d'un diagramme états-transitions UML.

Photos : Réglage de l'exposition On modélise ici les différents modes de réglage de l'exposition sur un appareil photo : mode auto-focus (AF), mode manuel avec priorité à l'ouverture (PO) et mode manuel avec priorité à la vitesse (PV).

Par défaut, l'appareil est en mode AF. Le bouton `man` passe dans un des modes `Manuel`, par défaut en mode PV ou dans le dernier mode manuel sélectionné. On peut passer du mode PO au mode PV, et vice versa, à l'aide du bouton `man`. En mode PO ou PV, le bouton `af` refait passer en mode auto focus. En mode AF, on peut aussi activer le mode 'macro' à l'aide du bouton du même nom, ce qui fait passer en mode PO.

Modélisez la gestion de ces modes sous forme d'un diagramme états-transitions UML.

Photos : Utilisation du flash L'objectif est de modéliser les différentes possibilités de réglage du flash d'un appareil photo, combiné avec le mode d'atténuation de l'effet « yeux rouges ».

Il y a deux grands modes de fonctionnement : sans flash ou avec flash. L'activation du mode « yeux rouges » n'est possible que quand le flash est actif. Au démarrage, l'appareil est réglé en mode 'flash Auto' et 'atténuation yeux rouges', noté `Attenuation YR`. Lorsque le flash est actif, on peut activer et désactiver le mode `Attenuation YR`. On passe de l'un à l'autre à l'aide du bouton `yr`. Trois modes de fonctionnement du flash sont possibles : 'flash auto', 'flash forcé' et 'flash doux'. On passe de l'un à l'autre, et dans cet ordre, par appui sur le bouton `flash`. On passe dans le mode 'sans flash' avec le bouton `no flash`. Dans ce mode, on peut réactiver le flash (bouton `flash`), ce qui fait passer en mode 'flash forcé' et 'atténuation yeux rouges'.

Modélisez la gestion de ces modes sous forme d'un diagramme états-transitions UML.

La vie des animaux Le comportement d'un lion est le suivant, méthode `live()`.

1. Par défaut, un lion se repose.
2. Quand il a faim (`isHungry():boolean {query}`), il chasse.
3. S'il est épuisé (`isExhausted():boolean {query}`), il se repose à nouveau.
4. Quand un lion chasse et qu'il capture un proie, il la mange (`eatAnimal()`) et il se repose ensuite.

Représentez ce comportement de la classe `Lion` par un diagramme états-transitions.

On va maintenant raffiner le comportement de chasse des lions.

1. Le lion regarde s'il y a une proie dans son voisinage.
2. S'il voit une proie, il se déplace dans la case correspondante, sinon, il se déplace aléatoirement.
3. Si la proie est toujours dans la case, il la capture ; si la proie n'y est plus, le lion regarde à nouveau dans le voisinage.

Modélisez ce comportement : identifiez les opérations correspondantes de la classe `Lion` et l'enchaînement possible de leur réalisation sous forme d'un diagramme états-transitions.

1.4 Mise en œuvre

1.4.1 Un modèle d'états-transitions pour quelles classes ?

Toutes les classes ne requièrent pas la modélisation de leur comportement. Il s'agit d'identifier celles qui ont un comportement complexe :

- ▷ Les objets de la classe réagissent différemment à l'occurrence du même événement
 - ◊ Chaque type de réaction caractérise un état particulier
- ▷ La classe doit réaliser certaines opérations dans un ordre précis
 - ◊ Des états séquentiels permettent de préciser la chronologie forcée des opérations

1.4.2 Comment construire un diagramme d'états-transitions ?

4 étapes :

1. Représentez la séquence d'états qui décrit le comportement nominal d'un objet, avec les transitions associées
2. Ajoutez progressivement les transitions qui correspondent aux comportements alternatifs ou d'erreur
3. Complétez les activités sur les transitions et dans les états
4. Structurez le diagramme en sous-états s'il devient trop complexe

1.4.3 Quand utiliser les diagrammes d'états-transitions ?

Convient bien pour décrire le *comportement d'un objet* à travers plusieurs cas d'utilisation.
Convient mal pour décrire un comportement qui implique plusieurs objets qui collaborent

- Les diag. d'interaction décrivent bien le comportement de plusieurs objets dans un cas d'utilisation
- Les diag. d'activités montrent bien l'organisation séquentielle globale des activités de plusieurs objets et cas d'utilisation

2 Diagramme d'activités

Sommaire

2.1	Activités : introduction	34
2.2	Activités : concepts de base	34
2.2.1	Activité	34
2.2.2	Flot de contrôle et de données	39
2.3	Activités : concepts avancés	40
2.3.1	Introduction	40
2.3.2	Nœud initial, final et de terminaison	41
2.3.3	Nœuds de décision et de fusion	41
2.3.4	Débranchement et branchement	43
2.3.5	Exemple	44
2.3.6	Région d'expansion	44
2.3.7	Partition d'activités	46

2.1 Activités : introduction

Le diagramme d'activité propose une modélisation du comportement du système, en mettant l'accent sur les traitements effectués : les **activités du système**. Il peut être utilisé en modélisation :

- ▷ Processus métier : aspects conceptuels et organisationnels
Réalisation des cas d'utilisation
- ▷ Conception du logiciel : réalisation des actions de base et structures de contrôle
Algorithmique des méthodes

Ce diagramme met l'accent sur le **flux de contrôle** et le **flux de données** d'une activité à l'autre.

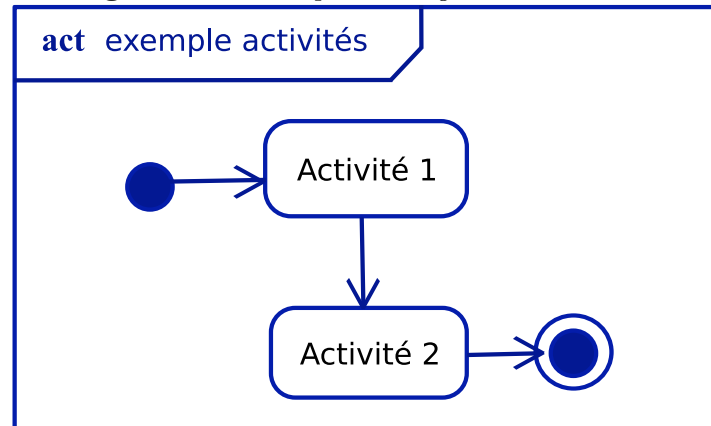
2.2 Activités : concepts de base

2.2.1 Activité

Définition Une activité est un élément de comportement d'un classier. Son traitement est exécutable par une entité du système. L'exécution d'une activité conduit à l'exécution d'actions en respectant un certain séquençement. Dans ce modèle, l'exécution d'une activité n'est pas instantanée et n'est pas atomique. Des activités peuvent s'exécuter en **parallèle**.

Notation

Figure 2.1. Exemple de diagramme d'activité



Action Une action est une *unité fondamentale d'exécution d'une activité* : plus petit élément de modélisation d'un comportement en UML. Elle est exécutée dans le *contexte* d'un classier. L'exécution d'une action correspond à une *transformation d'éléments du système* :

- ▷ Syst. informatique : lecture / modification de variables ou de propriétés du classier, appel d'une opération, création de nouveaux objets
- ▷ Processus métier : traitement d'une information, valeur ajoutée au processus

Une action est caractérisée par un *effet* : *description textuelle* des conséquences de son exécution sur les éléments du système.

Une action contient les propriétés suivantes :

- ▷ **Contraintes** : conditions devant être satisfaites
(ne pas confondre avec une condition de garde)
 - ◇ *localPrecondition* : quand l'action démarre
 - ◇ *localPostcondition* : quand l'action est terminée

Leur traitement est un point de variation sémantique

- ▷ **Entrées et Sorties** : *pins*
ensembles ordonnés de variables nommées
 - ◇ *input pin*

◇ *output pin*

▷ **Exception(s)** : *exception pin*

Si une exception survient, l'action est abandonnée
et aucune sortie n'est générée

Action de communication

▷ *accept event* :

son exécution bloque le flot d'exécution tant qu'un événement n'est pas détecté, généralement un type de signal

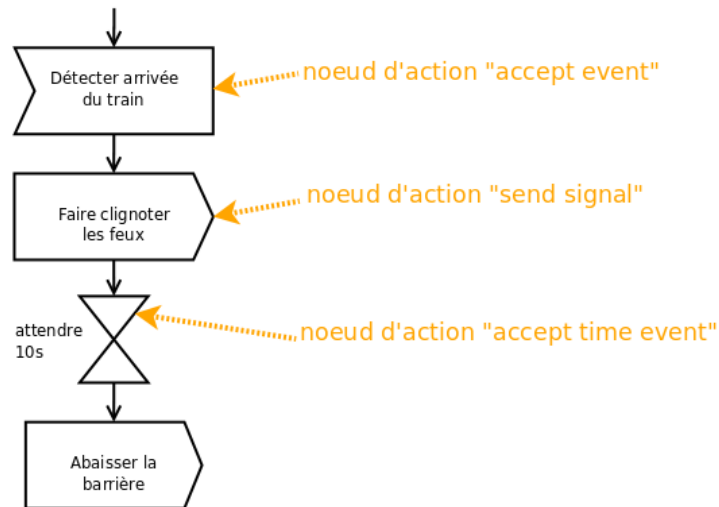
▷ *accept time event* :

attente qu'une expression temporelle se vérifie Ex. : "10 s", "tous les jours"

▷ *send signal* :

création d'un message et transmission de ce signal au destinataire

Figure 2.2. *Exemple d'action de communication*

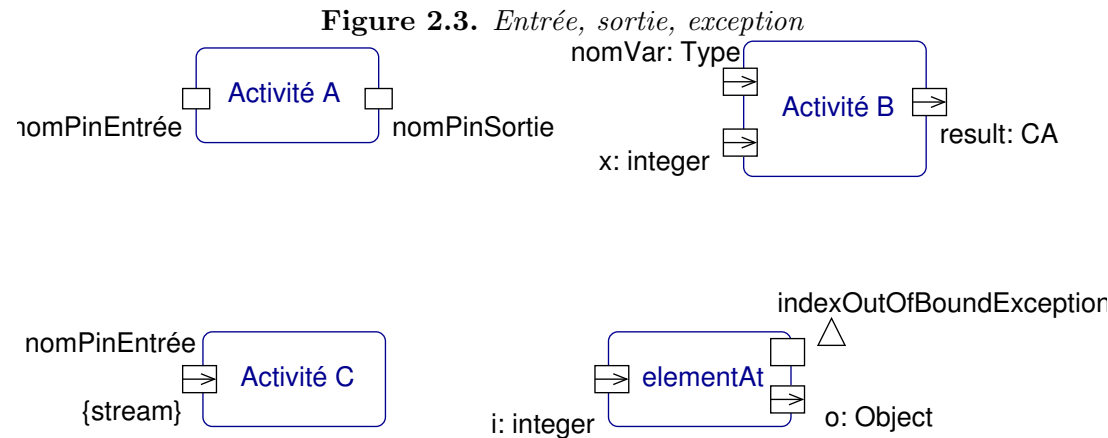


Pins Pour spécifier les valeurs passées en argument à une activité et les valeurs de retour, on utilise des noeuds d'objets appelés pins (pin en anglais) d'entrée ou de sortie. L'activité ne peut

débuter que si l'on affecte une valeur à chacun de ses pins d'entrée. Quand l'activité se termine, une valeur doit être affectée à chacun de ses pins de sortie.

Les valeurs sont passées par copie : une modification des valeurs d'entrée au cours du traitement de l'action n'est visible qu'à l'intérieur de l'activité.

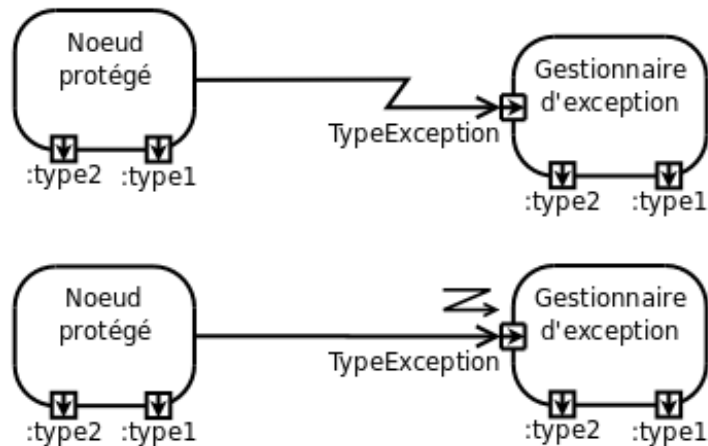
Graphiquement, un pin est représenté par un petit carré attaché à la bordure d'une activité. Il est typé et éventuellement nommé. Il peut contenir des flèches indiquant sa direction (entrée ou sortie) si l'activité ne permet pas de le déterminer de manière univoque.



Exception Une exception est générée quand une situation anormale entrave le déroulement nominal d'une tâche. Elle peut être générée automatiquement pour signaler une erreur d'exécution (débordement d'indice de tableau, division par zéro...), ou être soulevée explicitement par une action pour signaler une situation problématique qui n'est pas prise en charge par la séquence de traitement normale. Graphiquement, on peut représenter le fait qu'une activité peut soulever une exception comme un pin de sortie orné d'un petit triangle et en précisant le type de l'exception à proximité du pin de sortie.

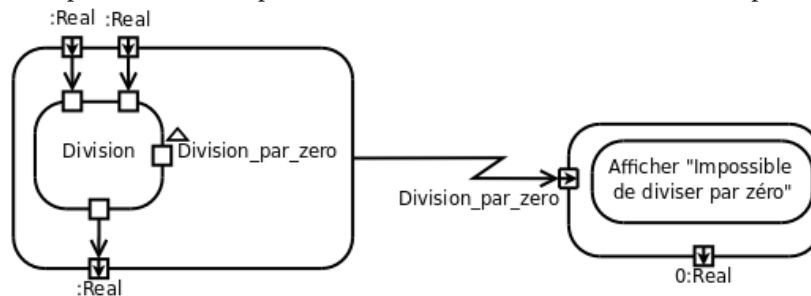
Un gestionnaire d'exceptions est une activité possédant un pin d'entrée du type de l'exception qu'il gère et lié à l'activité qu'il protège par un arc en zigzag ou un arc classique orné d'une petite flèche en zigzag. Le gestionnaire d'exceptions doit avoir les mêmes pins de sortie que le bloc qu'il protège.

Figure 2.4. *Les deux notations de la connexion entre une activité / gestionnaire d'exceptions*



Les exceptions sont des classeurs et, à ce titre, peuvent posséder des caractéristiques comme des attributs ou des opérations. Il est également possible d'utiliser la relation d'héritage sur les exceptions. Un gestionnaire d'exceptions spécifie toujours le type des exceptions qu'il peut traiter, toute exception dérivant de ce type est donc également prise en charge.

Figure 2.5. Exemple d'utilisation d'un gestionnaire d'exceptions pour protéger une activité de l'exception *Division_par_zéro* déclenchée en cas de division par zéro



Lorsqu'une exception survient, l'exécution de l'activité en cours est abandonnée sans générer de valeur de sortie. Le mécanisme d'exécution recherche alors un gestionnaire d'exceptions susceptible de traiter l'exception levée ou une de ses classes parentes. Si l'activité qui a levé l'exception n'est pas protégée de cette exception, l'exception est propagée à l'activité englobante.

L'exécution de cette dernière est abandonnée, ses valeurs de sortie ne sont pas générées et un gestionnaire d'exception est recherché à son niveau. Ce mécanisme de propagation se poursuit jusqu'à ce qu'un gestionnaire adapté soit trouvé. Si l'exception se propage jusqu'au sommet d'une activité (i.e. il n'y a plus d'activité englobante), trois cas de figure se présentent. Si l'activité a été invoquée de manière asynchrone, aucun effet ne se produit et la gestion de l'exception est terminée. Si l'activité a été invoquée de manière synchrone, l'exception est propagée au mécanisme d'exécution de l'appelant. Si l'exception s'est propagée à la racine du système, le modèle est considéré comme incomplet ou mal formé. Dans la plupart des langages orientés objet, une exception qui se propage jusqu'à la racine du programme implique son arrêt. Quand un gestionnaire d'exceptions adapté a été trouvé et que son exécution se termine, l'exécution se poursuit comme si l'activité protégée s'était terminée normalement, les valeurs de sortie fournies par le gestionnaire remplaçant celle que l'activité protégée aurait dû produire.

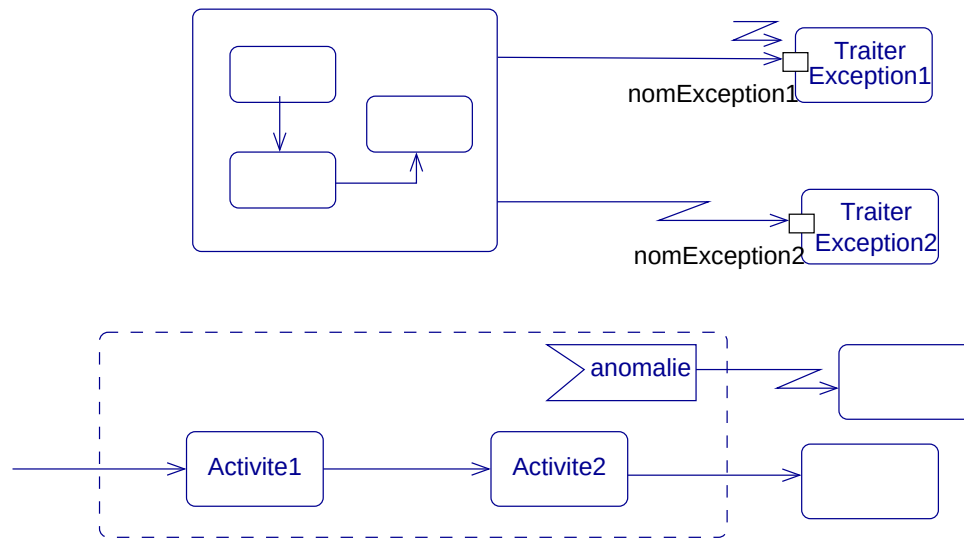


Figure 2.6.

2.2.2 Flot de contrôle et de données

Définition Deux approches sont possibles :

- ▷ Flot de contrôle : description des séquences d'exécution des activités

- ▷ Flot de données : passage d'une information produite par une activité à une activité qui la consomme

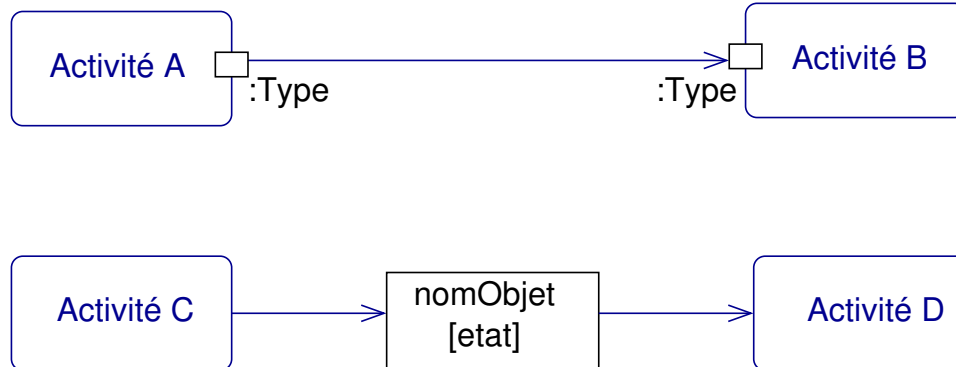
Exécution d'une action L'exécution d'une action « consomme » les flots de contrôle et d'objets entrants, elle ne peut donc commencer que s'ils sont valides. Quand elle est terminée, elle valorise ses flots de données de sortie (sauf si exception) et « passe » le contrôle aux activités cibles.

Objet d'un flot de données L'objet d'un flot de données est un élément d'information véhiculé par un **flot de données**. Il peut informer :

- ▷ instance d'un classier (obligatoire)
- ▷ état(s) du classier requis à ce point de l'exécution

Notation

Figure 2.7. Deux notations possibles



2.3 Activités : concepts avancés

2.3.1 Introduction

Il existe plusieurs types de nœuds de contrôle :

- ▷ Nœud initial, final et de terminaison

- ▷ Nœud de décision et nœud de fusion (structure conditionnelle)
- ▷ Nœud de débranchement et de jointure (*fork – join*)

2.3.2 Nœud initial, final et de terminaison

Nœud initial indique la première activité à réaliser : initialisation du flot de contrôle

notation : idem état–transition

Nœud final

- ▷ **Activité** (*activity final node*) :
indique que le flot de contrôle est terminé : TOUS les flots sont alors désactivés
notation : idem état–transition
- ▷ **Flot** (*flow final node*) :
UN des flots de contrôle s’achève (cas parallélisme)
notation : point de sortie états-transitions $\otimes$

2.3.3 Nœuds de décision et de fusion

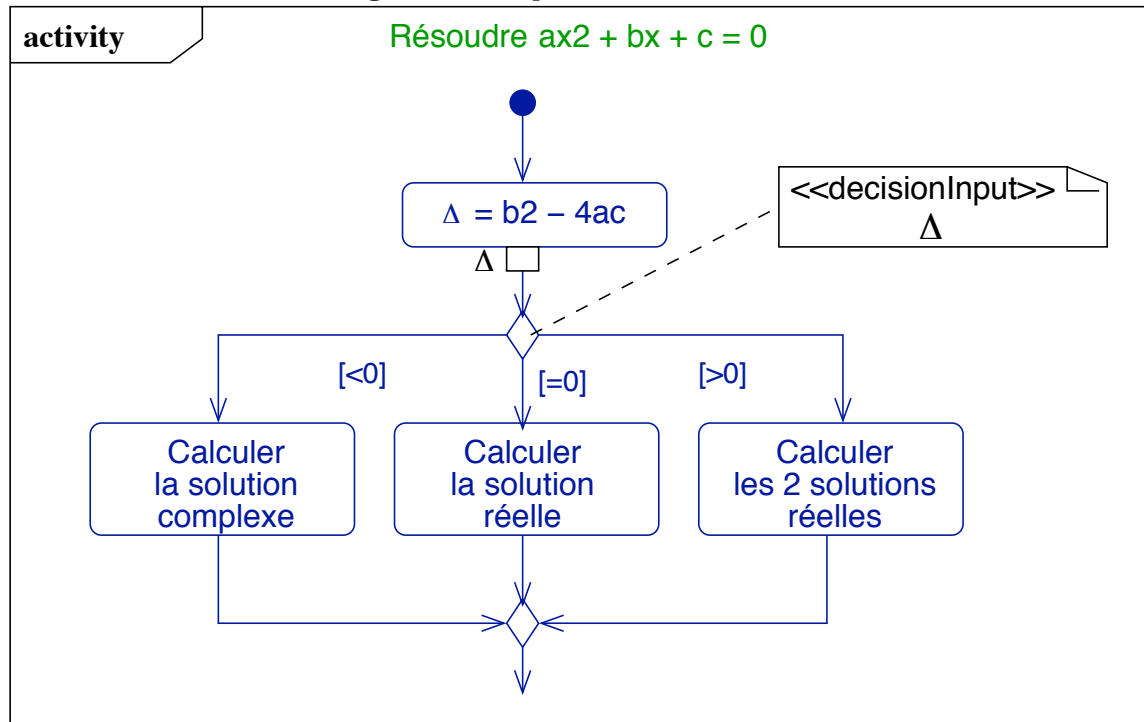
Nœud de décision

- ▷ Choix exclusif entre plusieurs membres d’une alternative
- ▷ 1 arc entrant et plusieurs arcs sortants (1 par choix)
- ▷ 1 *DecisionInput* $\equiv$ « règle de décision »
- ▷ Chaque arc sortant correspond à une valeur possible de *decision input* (ou [else])
expression du même type que *DecisionInput*
- ▷ Dans un modèle bien formé, l’alternative couvre tous les cas

Nœud de fusion : point où des chemins alternatifs se rejoignent

Exemple

Figure 2.8. Equation $ax^2 + bx + c = 0$



2.3.4 Débranchement et branchement

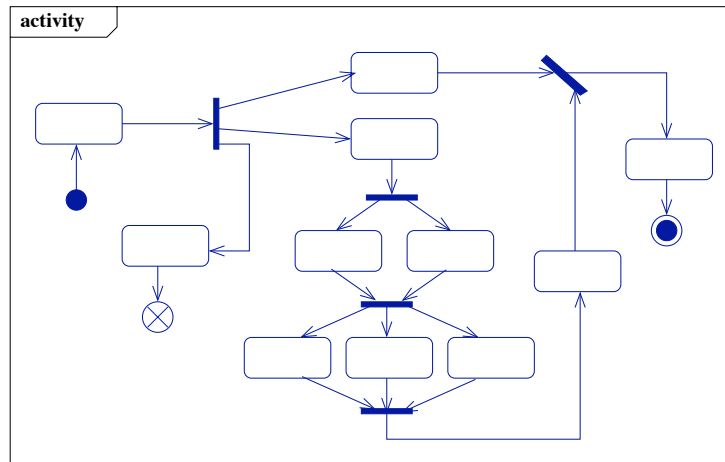


Figure 2.9.

2.3.5 Exemple

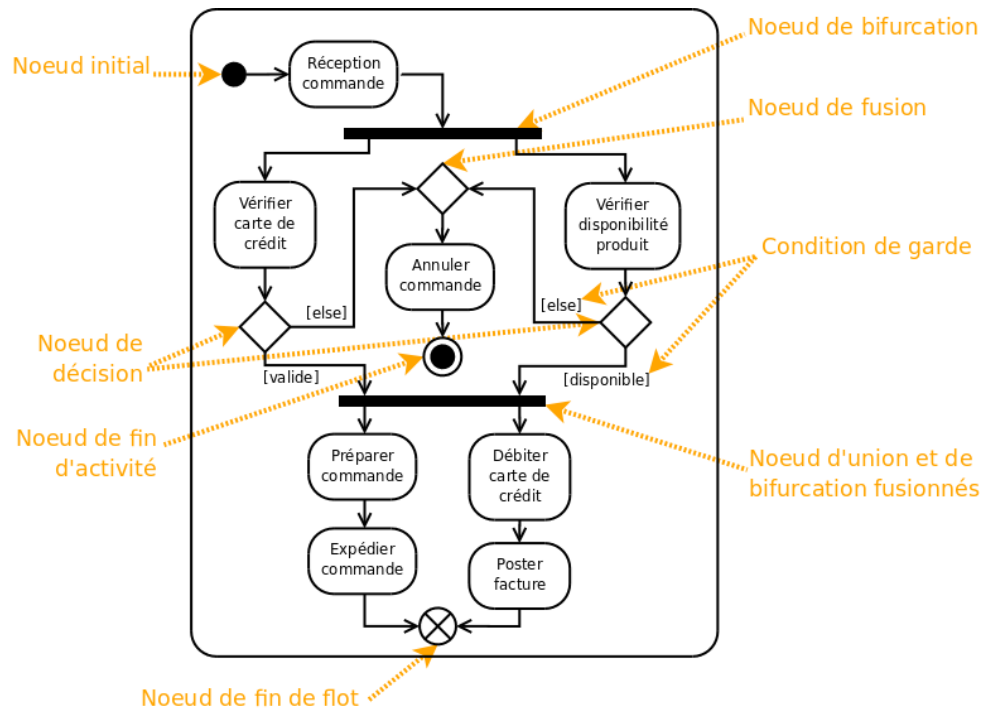


Figure 2.10.

2.3.6 Région d'expansion

Sémantique de l'exécution

- ▷ **Noeud d'expansion** : collection d'éléments traités par les activités de la région.
 - ◊ Noeud d'expansion d'entrée : la collection est produite par une activité en amont ; les activités de la région d'expansion traitent chaque élément de la collection
 - ◊ Noeud d'expansion de sortie : chaque exécution de la région produit un élément de la collection
- ▷ **Modes d'exécution** :

- ◇ «parallel» : de manière indépendante
- ◇ «iterative» : nécessairement en séquence
si collection ordonnée, alors dans l'ordre
- ◇ «streaming» : une exécution unique pour tous les éléments de la collection

Exemple Une région d'expansion permet l'exécution d'une activité sur les éléments d'une collection.

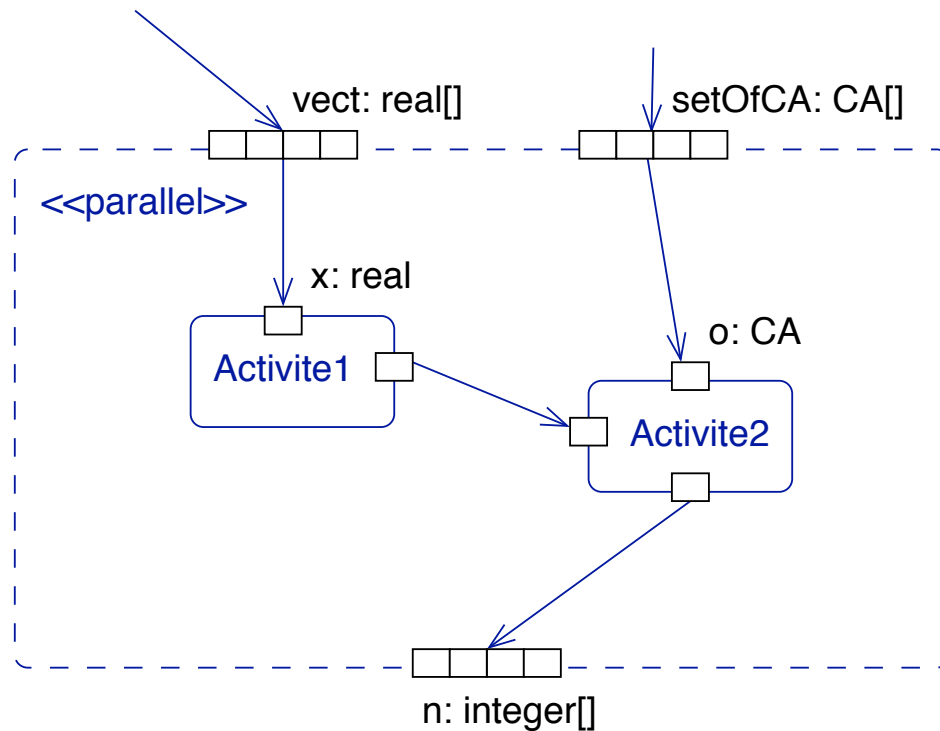


Figure 2.11.

Ce schéma correspond au cas particulier où les éléments des collections correspondant aux nœuds d'expansion d'entrée peuvent être traités indépendamment, ce qu'exprime le stéréotype «parallel».

Sur cet exemple, il y a deux collections d'objets en entrée et une collection de sorties. Ces collections doivent nécessairement contenir le même nombre d'élément : il y a une exécution pour chaque couple d'éléments des deux collections et chaque exécution produit un élément de la collection de sortie.

2.3.7 Partition d'activités

Définition Les partitions, souvent appelées couloirs ou lignes d'eau (swimlane) du fait de leur notation, permettent d'organiser les nœuds d'activités dans un diagramme d'activités en opérant des regroupements.

Les partitions n'ont pas de signification bien arrêtée, mais correspondent souvent à des unités d'organisation du modèle. On peut, par exemple, les utiliser pour spécifier la classe responsable de la mise en œuvre d'un ensemble de tâches. Dans ce cas, la classe en question est responsable de l'implémentation du comportement des nœuds inclus dans ladite partition.

Graphiquement, les partitions sont délimitées par des lignes continues. Il s'agit généralement de lignes verticales, mais elles peuvent être horizontales ou même courbes. Dans la version 2.0 d'UML, les partitions peuvent être bidimensionnelles, elles prennent alors la forme d'un tableau. Dans le cas d'un diagramme d'activités partitionné, les nœuds d'activités appartiennent forcément à une et une seule partition. Les transitions peuvent, bien entendu, traverser les frontières des partitions.

Les partitions d'activités étant des catégories arbitraires, on peut les représenter par d'autres moyens quand une répartition géométrique s'avère difficile à réaliser. On peut ainsi utiliser des couleurs ou tout simplement étiqueter les nœuds d'activité par le nom de leur partition d'appartenance.

Exemples

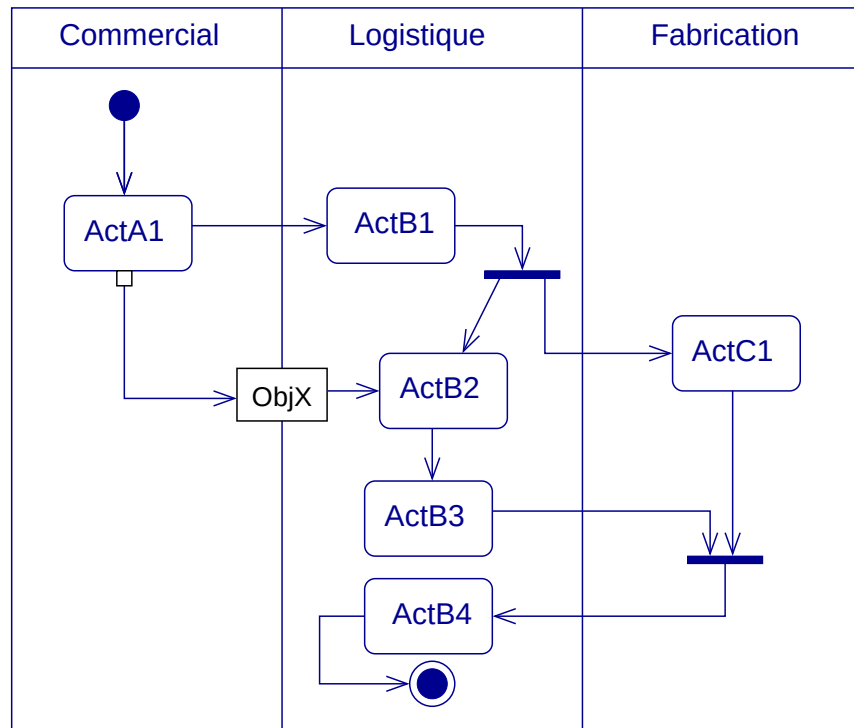


Figure 2.12.

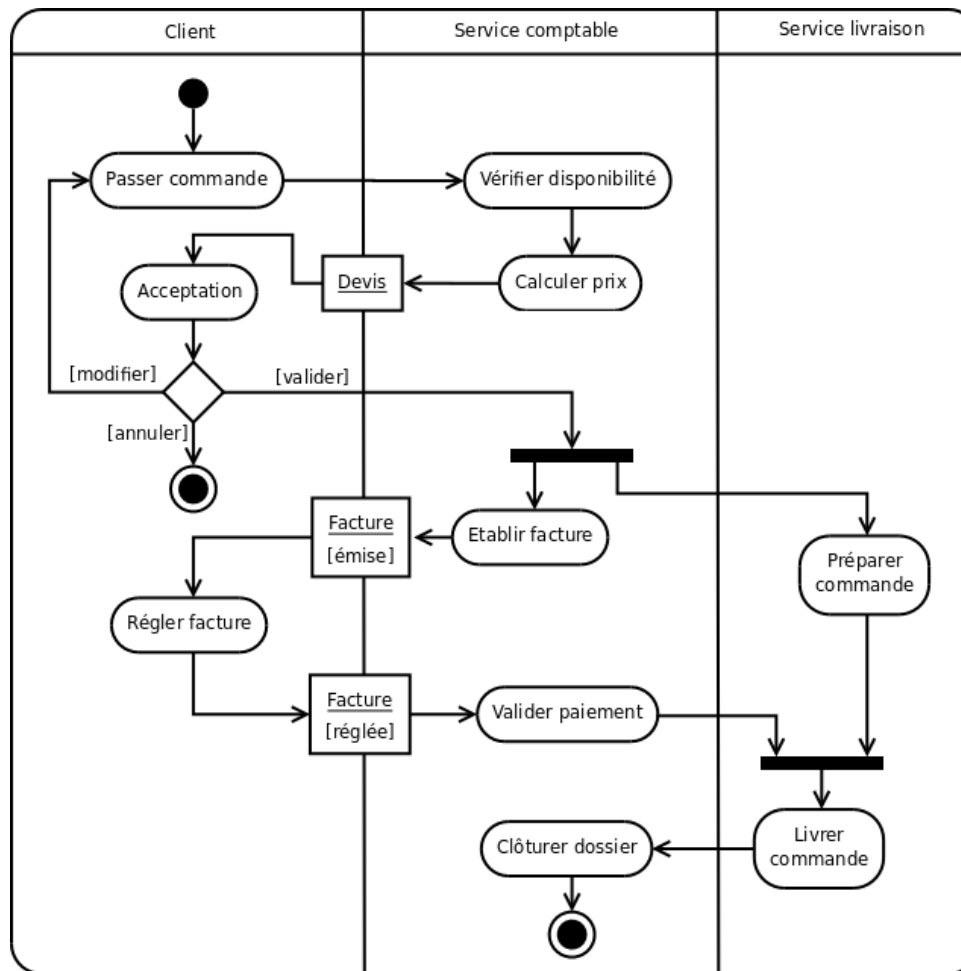


Figure 2.13.

3 Génération de code

Sommaire

3.1	Diagramme de classes	50
3.1.1	Principe	50
3.1.2	Package	50
3.1.3	Classes	51
3.1.4	Opération	52
3.1.5	Types	53
3.1.6	Attributs	55
3.1.7	Instanciation	61
3.1.8	Association	65
3.1.9	Autres relations	75
3.1.10	Exercices	78
3.2	Diagramme de séquences	81
3.3	Diagramme de communication	82
3.4	Diagramme d'états-transitions	83

3.1 Diagramme de classes

3.1.1 Principe

UML est indépendant des langages de programmation et des technologies. Comment implémenter (=coder) les modèles? Certains éléments de modélisation sont communs à UML et aux LPO :

- ▷ Ont-ils la même sémantique?
- ▷ Sinon, comment assurer la traduction de l'un dans l'autre?
- ▷ Existe-t'il une et une seule traduction possible?
- ▷ La traduction est-elle « réversible »?

Il faut définir des *profiles* adaptés.

3.1.2 Package

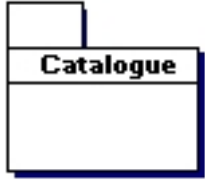
- ▷ Soit un Package de nom "pkgName"
- ▷ Dans le répertoire du contexte courant, création d'un répertoire pkgName (**Java** et **C++**)
- ▷ **Java** (dans chaque fichier correspondant aux classes du package) : `package pkgName;`
- ▷ **C++** :
 - ◇ Un répertoire pkgName dans l'arborescence `include`
 - ◇ Un répertoire pkgName dans l'arborescence `src`
 - ◇ Une bibliothèque dans `lib` (p. ex. `libpkgName.so`)
 - ◇ `namespace pkgName {...};`

Dépendance

- ▷ **Java** : `import pkgName.*;`
- ▷ **C++** : `using namespace pkgName;`

Définition 3.1. *LPO : langage de programmation (orienté) objet.*

Exemples de langage : C++, java, C#, Small-talk, Objective C...

Conversion	UML	Java
		<pre>package Catalogue ...</pre>
		C++
		<pre>namespace Catalogue { ... }</pre>

3.1.3 Classes

Soit une Class CName du package pkgName

- ▷ Un fichier par classe
(sauf pour les classes contenues dans une autre classe)
- ▷ **Java** : fichier CName.java dans le répertoire du package

```
package pkgName;
public class CName {...}
```
- ▷ **C++** :
 - ◊ CName.h dans l'arborescence include (rép. du pkgName)
 - ◊ CName.cpp dans l'arborescence src (rép. du pkgName)
 - ◊ fichier CName.o (bibliothèque libPkgName.so)
 - ◊ namespace pkgName {


```
class CName {...};
```

 }

Cas particuliers

- ▷ Classe abstraite
 - ◊ **Java** :


```
public abstract class AA { ... }
```

- ◇ C++ :
au moins une opération abstraite

▷ Interface :

- ◇ Java :
public interface IJ { ... }
- ◇ C++ :
classe dont toutes les opérations sont abstraites

En C++, une classe *interface* n'a aucune variable membre non statique, ni de constructeur explicite et ses fonctions membres sont toutes virtuelles pures ; comme toujours, le destructeur est virtuel.

3.1.4 Opération

▷ *Operation vs Method*

- ◇ Operation : spécification du service (déclaration)
- ◇ Method : définition de l'opération
- ◇ Une Operation sans Method associée est *abstraite*

▷ *Paramètres* :

- ◇ Type : voir traduction des types de base
- ◇ Direction : in, out, inout, return

3.1.5 Types

Java

▷ « type » integer – TaggedValue {size}, {wrapped}

	—	{wrapped}
—	int	Integer
{size,long}	long	Long
{size,short}	short	Short
{size,byte}	byte	Byte

▷ « type » real – TaggedValue {size}, {wrapped}

	—	{wrapped}
—	float	Float
{size,long}	double	Double

Remarque : il s'agit d'une possibilité de traduction, les TaggedValue {size} et {wrapped} ne font pas partie de la norme UML.

▷ « type » boolean – TaggedValue {wrapped}

—	{wrapped}
boolean	Boolean

▷ « type » string

String

▷ « type » char – TaggedValue {wrapped}

—	{wrapped}
char	Character

C++

▷ << type >> integer – TaggedValue {size}, {unsigned}

	—	{unsigned}
—	int	unsigned int
{size,long}	long	unsigned long
{size,extraLong}	long long	unsigned long long
{size,short}	short	unsigned short

▷ << type >> real – TaggedValue {size}

—	float
{size,long}	double
{size,extraLong}	long double

▷ << type >> boolean :
bool

▷ << type >> string :
string
#include <string>

▷ << type >> char :
char

3.1.6 Attributs

Principes généraux de la transformation

- ▷ *implémentation non accessible* en dehors de la classe
- ▷ *accessibilité* (lecture, modification) assurée par des méthodes dont l'accessibilité correspond à la *visibilité du modèle de classe*

Les principes de la transformation sont communs à tous les langages objets typés et supportant le principe d'encapsulation.

C++ Exemple d'un attribut `attrName`: `AttrType` protégé

```
class CName {
protected:
    inline AttrType getAttrName(void) const;
    inline void setAttrName(AttrType value);
    inline virtual void _invariant(void);

private:
    AttrType _attrName;
};

inline AttrType CName::getAttrName(void) const {
    return _attrName;
}
inline void CName::setAttrName(AttrType value) {
    _attrName = value;
    _invariant();
}

inline void CName::_invariant(void) {
    // Begin
    // End
}
```

Java Exemple de la classe Plant

+quantity: real = 0
«invariant» {quantity ∈ [0, maxQuantity]}

▷ *Implémentation privée* (cf. règle de codage) :

```
private double quantity;
```

▷ *Accesseur public* (cf. modèle) :

```
public double getQuantity() {  
    return quantity;  
}
```

▷ *Modificateur* : visibilité et maintien de l'invariant

```
public void setQuantity(double quantity) {  
    quantity = Math.max(quantity, 0.0);  
    quantity = Math.min(quantity, maxQuantity);  
    this.quantity = quantity;  
}
```



```
+growthRate: real = 0.01 {readOnly}
```

```
public Plant(..., double growthRate) {  
    super(...);  
    this.growthRate = growthRate;  
}  
  
public Plant(...) {  
    this(..., 0.01 /* initial value of growthRate */);  
}  
  
private final double growthRate;  
  
public double getGrowthRate() { return growthRate; }
```

+maxQuantity: real = 1.0 {readOnly}

```
private static double maxQuantity = 1;

public static double getMaxQuantity() {
    return maxQuantity;
}
```

```
# birthdate: real = Clock.time {readOnly}
+age: real = Clock.time - birthdate
```

```
private double birthdate;

protected double getBirthdate() {
    return birthday;
}

// — Derived attribute 'age'
public double getAge() {
    return Clock.getTime() - this.getBirthdate();
}
```

Java : attribut à multiples valeurs : Exemple de la classe SquareCell

+position: real[2]

▷ Traduction de real sous forme d'un *wrapper* :

```
private Vector<Double> position = new Vector<Double>(2);
public SquareCell(double x, double y) {
    position.set(0, x);
    position.set(1, y);
}
```

▷ Traduction de real sous forme d'un **type de base** :

```
private double[] position = new double[2];
public SquareCell(double x, double y) {
    position[0] = x;
    position[1] = y;
}
```

Masquage de l'implémentation

▷ *wrapper* :

```
public double getPosition(int index) {  
    return position.elementAt(index);  
}  
public void setPosition(int index, double value) {  
    position.set(index, value);  
}
```

▷ type de base :

```
public double getPosition(int index) {  
    return position[index];  
}  
void setPosition(int index, double value) {  
    position[index] = value;  
}
```

3.1.7 Instanciation

Instanciation d'une classe UML : constructeurs

▷ *Constructeurs implicites* :

- ◊ nom de la méthode : nom de la classe
- ◊ valeurs initiales des attributs
- ◊ construction des conteneurs pour l'implémentation des attributs à valeurs multiples et des associations de multiplicité > 1
- ◊ action(s) des états initiaux (cf. modèle état-transition)
- ◊ Cohérence des constructeurs

▷ *Constructeurs explicites* :

Operation stéréotypée `<<create>>`

Java exemple de la classe Clock : constructeurs

```
package simLife;

public class Clock {

    public Clock(double initialTime, double deltaT, double timeMax) {
        this.time = initialTime;
        this.deltaT = deltaT;
        this.timeMax = timeMax;
        // ...
    }

    public Clock(double initialTime, double deltaT) {
        this(initialTime, deltaT, Double.MAX_VALUE);
    }

    public Clock() {
        this(0.0, 1.0, Double.MAX_VALUE);
    }
}
```

C++ exemple de la classe CName : constructeurs

Soit CName ayant deux attributs :

attr1: integer = 0

attr2: string = ""

```
// File: CName.h
...
class CName {
public:
    CName(void);
    CName(const CName & instanceOfCName);
    const CName & operator = (const CName & instanceOfCName);
    virtual ~ CName(void);
    ...
protected:
    virtual void _copy(void);
    ...
};
```

// CName.cpp

```
CName::CName(void): attr1(0), attr2("") {  
    // Begin  
    // End  
}  
  
CName:: CName(const CName & instanceOfCName) {  
    _copy(instanceOfCName);  
}  
  
const CName & CName::operator = (const CName & instanceOfCName) {  
    if (this != & instanceOfCName) _copy(instanceOfCName);  
    return * this;  
}  
  
CName::~~ CName(void) {  
    // Begin  
    // End  
}  
  
void CName::_copy(const CName & instanceOfCName) {  
    this->setAttr1(instanceOfCName.getAttr1());  
    this->setAttr2(instanceOfCName.getAttr2());  
}
```


3.1.8 Association

Java Association unidirectionnelle en Java :Exemple de CName1 vers CName2

▷ Multiplicity = 0..1 (p. ex. visibilité public)

```
class CName1 {  
  
    // — Implémentation cachée de l'objet en association  
    private CName2 roleName;  
  
    // — Access public à l'objet en association  
    public CName2 getRoleName() {  
        return this.roleName;  
    }  
  
    // — Modification possible de la référence de l'objet en association  
    public void setRoleName(CName2 instance) {  
        if (!this.getRoleName().equals(instance))  
            this.roleName = instance;  
    }  
}
```

▷ Multiplicity = 0..* ou 1..* (p. ex. visibilité public)

```
class CName1 {  
    // — Implémentation cachée de la collection (= ensemble)  
    private Set<CName2> roleName = new HashSet<CName2>();  
  
    // — Access public à l'ensemble d'objets en association  
  
    public Set<CName2> getRoleName() {  
        return new Set<CName2>(roleName); }  
    public void setRoleName(Set<CName2> setOfCName2) { ... }  
  
    // — Modification de l'ensemble d'objets en association  
  
    public boolean addToRoleName(CName2 instance) { ... }  
    public boolean removeFromRoleName(CName2 instance) { ... }  
  
    // — Informations sur l'ensemble d'objets en association  
  
    public boolean hasInRoleName(CName2 instance) { ... }  
    public int cardOfRoleName() { ... }  
}
```

C++

▷ Multiplicity = 0..1

```
class CName1 {
public:
    inline virtual const CName2 * getRoleName(void) const;
    inline virtual void setRoleName(CName2 * instance);
private:
    CName2 * _roleName;
};

inline const CName2 * CName1::getRoleName(void) const {
    return _roleName;
}

inline void CName1::setRoleName(CName2 * instance) {
    if (getRoleName() != instance) {
        _roleName = instance;    _invariant();
    }
}
```

Multiplicity = 0..* ou 1..*

```
class CName1 {
public:

// — Global access to the set of instances 'roleName'
inline virtual const set < CName2 * > & getRoleName(void) const;

inline virtual void setRoleName(const set < CName2 * > & setOfInstance);

// — Add/remove one instance in/from 'roleName'
inline virtual bool addToRoleName(CName2 & instance);

inline virtual bool removeFromRoleName(CName2 & instance);

// — Get information about the set 'roleName'
inline virtual bool hasInRoleName(const CName2 & instance) const ;

inline virtual unsigned int cardOfRoleName(void) const;

// — Hidden implementation of 'roleName'
private:
    set < CName2 * > _roleName;
};
```

Java Association bidirectionnelle en Java : Gestion de l'intégrité du référencement réciproque

exemple : Animal – Place

```
private Place location;

public Place getLocation() { return location; }

public void setLocation(Place location) {
    if (location != null)
        if (this.location != null && this.location != location)
            this.location.removeFromLocalPopulation(this);
        if (this.location != location) {
            this.location = location;
            location.addToLocalPopulation(this);
        }
    }
}
```

```
class Place {

    private Set<Animal> localPopulation = new HashSet<Animal>();

    public void addToLocalPopulation(Animal animal) {
        if (!this.localPopulation.contains(animal)) {
            this.localPopulation.add(animal);
            animal.setLocation(this);
        }
    }
}
```

Classe-Association en Java : Exemple de la relation prédateurs – proies (classe Species)

```
class Species {
    private Set<FoodLink> preys = new HashSet<FoodLink>();
    ...
    private Set<FoodLink> predators = new HashSet<FoodLink>();
    ...
}

class FoodLink {
    private Species prey;
    private Species predator;
    private double preference;
    ...
}
```

```
public boolean hasInPreys(Species prey) {
    FoodLink fl = new FoodLink(this, prey, 0);
    Iterator<FoodLink> iter = preys.iterator();
    boolean found = false;
    while (iter.hasNext() && !found) found = iter.next().equals(fl);
    return found;
}
```

```
public boolean addInPreys(Species prey, double preference) {
    // check that the species is not already referenced as a prey
    FoodLink fl = new FoodLink(this, prey, preference);
    Iterator<FoodLink> iter = preys.iterator();
    boolean found = false, added = false;
    while (iter.hasNext() && !found) found = iter.next().equals(fl);
    if (!found) added = preys.add(fl) && prey.predators.add(fl);
    return added;
}
```

```
public boolean hasInPredators(Species predator) {
    FoodLink fl = new FoodLink(predator, this, 0);
    Iterator<FoodLink> iter = predators.iterator();
    boolean found = false;
    while (iter.hasNext() && !found)
        found = iter.next().equals(fl);
    return found;
}
```

```
public class FoodLink {  
    public FoodLink(Species predator, Species prey) {  
        this.prey = prey;  
        this.predator = predator;  
    }  
  
    public FoodLink(Species predator, Species prey, double preference) {  
        this.prey = prey;  
        this.predator = predator;  
        this.preference = preference;  
    }  
  
    public boolean equals(Object o) {  
        if (!(o instanceof FoodLink)) return false;  
        FoodLink f = (FoodLink)o;  
        return f.prey == this.prey && f.predator == this.predator;  
    }  
}
```


Association qualifiée en Java : Exemple : Savanna – Species

```
private static Map<String, Species> species = new HashMap<String, Species>();

public static Species getSpecies(String name) {
    return Savanna.species.get(name);
}

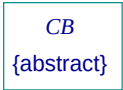
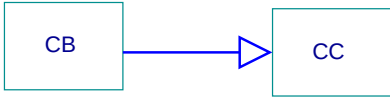
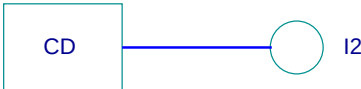
public static void addToSpecies(String name, Species species) {
    Savanna.species.put(name, species);
}

public static void removeFromSpecies(String name) {
    Savanna.species.remove(name);
}

public static boolean hasInSpecies(String name) {
    return Savanna.species.containsKey(name);
}

public static int numberOfSpecies() { return Savanna.species.size();}
```

Classes, interface, généralisation, implémentation

	java	<code>public class CA { /* */ }</code>
	C++	<code>class CA { /* */ };</code>
	java	<code>public abstract class CA { /* */ }</code>
	C++	<code>class CB { /* */ };</code>
	java	<code>public interface I1 { /* */ }</code>
	C++	<code>class I1 { /* */ };</code>
	java	<code>public class CB extends CC { /* */ }</code>
	C++	<code>class CB: public CC { /* */ };</code>
	java	<code>public class CD implements I2 { /* */ }</code>
	C++	<code>class CD: public I2 { /* */ };</code>

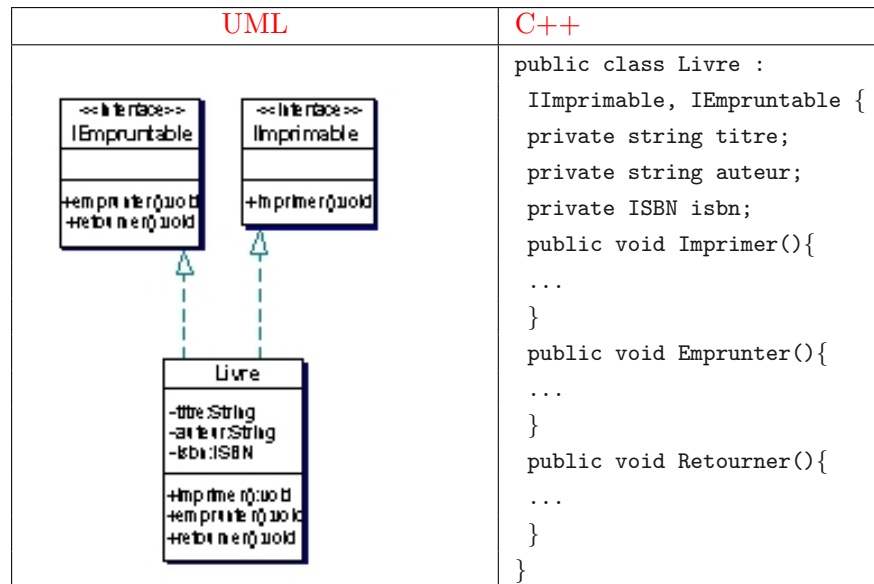
Les concepts de classe abstraite et d'interface n'existent pas en tant que tel en C++ ; cependant il est possible de traduire ces notions en classes C++ ayant des propriétés équivalentes.

3.1.9 Autres relations

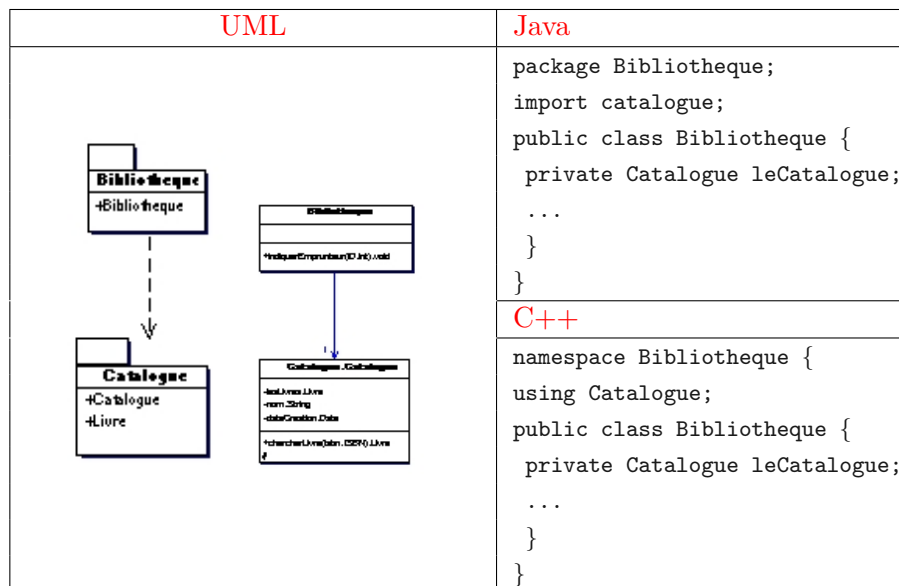
Réalisation en Java

UML	Java
<pre> classDiagram class IEmpruntable { +emprunter() void +retourner() void } class IImprimable { +Imprimer() void } class Livre { -titre: String -auteur: String -isbn: ISBN +Imprimer() void +Emprunter() void +Retourner() void } IEmpruntable < .. Livre IImprimable < .. Livre </pre>	<pre> public class Livre implements IImprimable, IEmpruntable { private String titre; private String auteur; private ISBN isbn; public void Imprimer(){ ... } public void Emprunter(){ ... } public void Retourner(){ ... } } </pre>

Réalisation en C++



Dépendance



Agrégation

UML	Java
<pre> classDiagram class Voiture { -modele String } class Moteur { -puissance int } Voiture "1" -- "*" Moteur </pre>	C++
	<pre> public class Voiture { private String modele; private Moteur moteur; private static class Moteur { private int puissance; } ... } </pre>

3.1.10 Exercices

Java => diagramme de classes On considère le code JAVA ci-dessous et pages suivantes.

Traduire ce code en un diagramme de classes UML.

```
// File: AA.java -----  
package pkgA;  
  
abstract public class AA {  
  
    public abstract float ope1();  
}  
// -----
```

```
// File: I1.java -----  
package pkgA;  
  
public interface I1  
{  
    boolean service1();  
}  
// -----
```

```
// File: BB.java -----
package pkgA;

public class BB implements pkgA.I1
{
    private int attr = 1;
    protected int getAttr() {
        return this.attr;
    }
    protected void setAttr(int value) {
        this.attr = value;
    }

    private pkgA.CC theCC;
    protected pkgA.CC getTheCC() {
        return this.theCC;
    }

    protected int cardTheCC() {
        if ( this.theCC == null ) return 0;
        else return 1;
    }

    public boolean service1() {
        // ...
    }
} // -----
```

```
// File: App.java -----
package pkgA;

public class App
{
    public static void main(final String[] args) {
        CC objCC = new CC();
    }
}
// -----
```

```
// File: ACC.java -----
package pkgA;

import java.util.*;

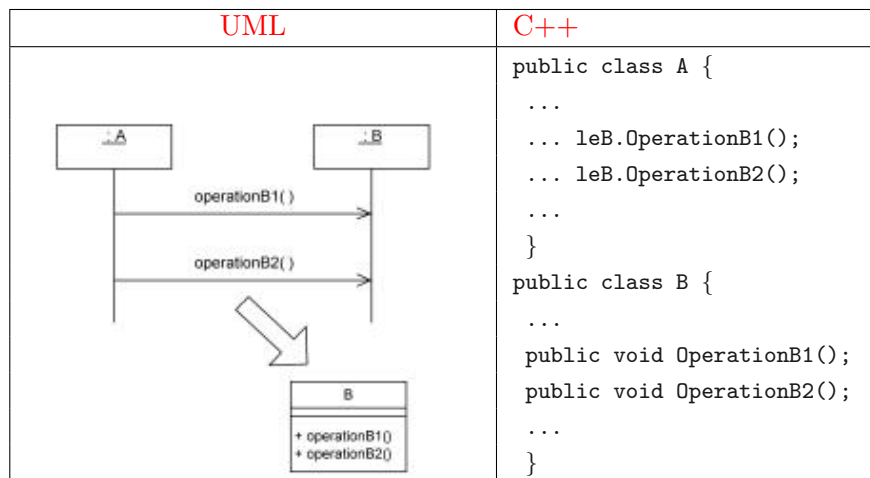
public class CC extends pkgA.AA
{
    private Vector someBB = new Vector();

    public pkgA.BB getSomeBB(int i) {
        return (pkgA.BB)this.someBB.elementAt(i);
    }
    public void setSomeBB(int i, pkgA.BB value) {
        this.someBB.setElementAt(value, i);
    }
    public void appendSomeBB(pkgA.BB value) {
        this.someBB.addElement(value);
    }
    public void eraseSomeBB(pkgA.BB value) {
        this.someBB.removeElement(value);
    }
    public void eraseSomeBB(int i) {
        this.someBB.removeElementAt(i);
    }
    public int cardSomeBB() {
        return this.someBB.size();
    }

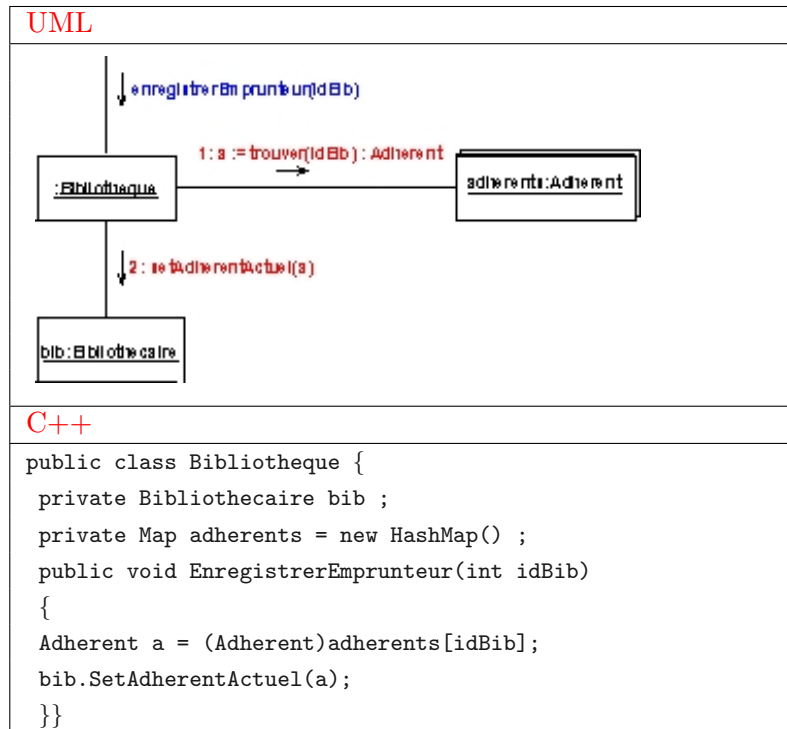
    public float ope1() {
        float result = 1.0;
        return result;
    }

    public int ope2(final float Parameter, pkgA.BB objBB) {
        int result = 1;
        objBB.service1();
        return result;
    }
}
// -----
```


3.2 Diagramme de séquences

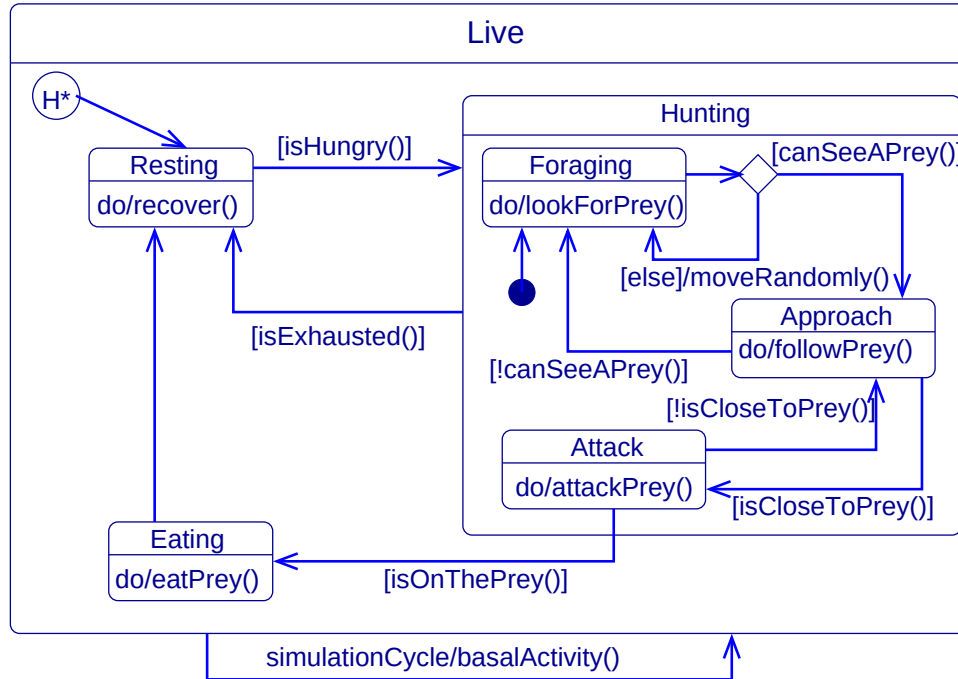


3.3 Diagramme de communication



3.4 Diagramme d'états-transitions

Figure 3.1. *Exemple Etat-Transition : comportement d'un Lion*



Propriétés de ce modèle :

- ▷ Pas d'événement sur les transitions « internes »
- ▷ Transitions gardées (éventuellement toujours vraie)
- ▷ Déterministe
- ▷ Pas de régions orthogonales
- ▷ Actions associées aux états (et aux transitions)

Implémentation procédurale en Java :

```

public class Lion extends Animal {

    public enum State {
        RESTING,
        HUNTING,
        FORAGING,
        APPROACH,
        ATTACK,
        EATING
    }

    Set<State> activeStates;

```

État initial (constructeur) :

```

public Lion(SquareCell location) {
    super(Savanna.getSpecies("Lion"), location);
    activeStates = new HashSet<State>();
    activeStates.add(State.RESTING); // initial state
}

```

```

public void live() {
    // Deep history
    // Do action corresponding to currently active state
    if (activeStates.contains(State.RESTING))
        this.recover();
    else if (activeStates.contains(State.HUNTING))
        this.hunting();
    else if (activeStates.contains(State.EATING))
        eatPrey();
    // ...
}

```

Gestion des transitions :

```
public void live() {  
    // ...  
    // Outgoing transitions  
    if (activeStates.contains(State.RESTING) && isHungry()) {  
        activeStates.remove(State.RESTING);  
        this.hunting();  
    } else if (activeStates.contains(State.EATING)) {  
        activeStates.remove(State.EATING);  
        activeStates.add(State.RESTING);  
    }  
}
```

Remarques sur ce modèle :

- ▷ Toutes les actions dont la réalisation est contrôlée par la machine à état sont des opérations **protected**
- ▷ la gestion de l'état composite *Hunting* est assurée par une méthode dédiée.

Motivations :

- ◊ Homogénéité de la transformation
- ◊ **Spécialisation comportementale** : cohérence avec le modèle de classe
- ▷ Les conditions définissant les gardes des transitions sont assurées par des opérations **protected** {isQuery}

Activation de l'état :

```
protected void hunting() {  
    // Entering the state (deep history)  
    if (! activeStates.contains(State.HUNTING)) {  
        activeStates.add(State.HUNTING);  
        activeStates.add(State.FORAGING);  
    }  
}
```

Activités internes :

```
// Do action corresponding to the current state
if (activeStates.contains(State.FORAGING))
    lookForPreys();
else if (activeStates.contains(State.APPROACH))
    followPrey();
else if (activeStates.contains(State.ATTACK))
    attackPrey();
```

Gestion des transitions sortantes :

```
// Outgoing transitions
if (activeStates.contains(State.HUNTING) && this.isExhausted()) {
    // quit the superstate (all active states)
    // and activate Resting
    activeStates.remove(State.HUNTING);
    activeStates.remove(State.FORAGING);
    activeStates.remove(State.APPROACH);
    activeStates.remove(State.ATTACK);
    activeStates.add(State.RESTING);
}
```

Gestion des transitions :

```

} else if (activeStates.contains(State.ATTACK) && isOnThePrey()) {
    activeStates.remove(State.HUNTING);
    activeStates.remove(State.FORAGING);
    activeStates.remove(State.APPROACH);
    activeStates.remove(State.ATTACK);
    activeStates.add(State.EATING);

} else if (activeStates.contains(State.FORAGING)) {
    if (canSeeAPrey()) {
        activeStates.remove(State.FORAGING);
        activeStates.add(State.APPROACH);
    } else {
        this.moveRandomly();
    }
}

} else if (activeStates.contains(State.APPROACH) && ! canSeeAPrey()) {
    activeStates.remove(State.APPROACH);
    activeStates.add(State.FORAGING);
} else if (activeStates.contains(State.APPROACH) && isCloseToPrey()) {
    activeStates.remove(State.APPROACH);
    activeStates.add(State.ATTACK);
} else if (activeStates.contains(State.ATTACK) && ! isCloseToPrey()) {
    activeStates.remove(State.ATTACK);
    activeStates.add(State.APPROACH);
}

```

Labo

4 En attendant ...

Sommaire

4.1	AReViLib3d	89
4.2	Mail 2	90
4.3	Voisinage Cellule	91
4.4	Album Photos	92

4.1 AReViLib3d

La conception d'une application graphique repose sur l'utilisation d'une bibliothèque de classes pour le rendu graphique 3D qui s'appelle ARéVi.

Dans cette bibliothèque, les objets graphiques sont de la classe `Object3D`. Les objets sont localisés dans l'espace, propriété qui est définie dans la classe `Base3D` dont hérite `Object3D`. La classe `Base3D` offre les services publics suivants :

1. `locate` : positionnement en un point donné spécifié par un ensemble de 3 coordonnées (nombres réels) ;
2. `move` : déplacement selon un vecteur spécifiée sous la forme d'une `Base3D` ;
3. `location` : obtention de coordonnées de l'objet, sous forme de 3 nombres réels ;
4. `attachTo` : attachement à une autre `Base3D` (en conséquence les 2 `Base3D` sont liées, le déplacement de l'une se répercute sur l'autre).

La classe `Object3D` offre des services qui permettent de définir comment un objet 3D réagit quand on interagit avec lui dont :

1. `onMouseInteraction` : définition du comportement quand on interagit avec l'objet à l'aide d'une souris. Cette opération utilise les services d'un `Interactor` qui est ici une interface. On lui fournit aussi le numéro du bouton actionné et on indique si le bouton est pressé (oui ou non).

L'interface `Interactor` définit le service `getEvent` qui fournit en sortie un objet de la classe abstraite `Event`. La classe concrète `BasicMouseInteractor` réalise cette interface.

Représentez toutes ces informations sous forme d'un modèle de classes UML.

La conception porte sur un objet complexe qui permet de représenter graphiquement des surfaces paramétrées. Ceci est supporté par un ensemble de classes regroupées dans le package **grapher** qui utilise les classes de la bibliothèque ARéVi (voir exercice correspondant).

La structure d'un grapher (classe **Grapher**) est la suivante :

1. il est composé de 3 axes, Ox , Oy et Oz ;
2. à chaque axe on associe un domaine de valeur qui est défini par un min, un max (nombres réels) et un nombre de valeurs ;
3. le grapher peut afficher une ou plusieurs surfaces (chaque surface référence le grapher dans lequel elle est représentée) ;
4. 2 domaines sont associés à chaque surface : x et y (ceci permet de savoir qu'elle partie de la surface est visible).

Les axes et les surfaces sont des **Object3D** particuliers, le grapher une **Base3D**.

Représentez toutes ces informations sous forme d'un modèle de classes UML.

4.2 Mail 2

La gestion locale des messages par l'utilisateur repose sur la notion de compte de messagerie (souvent appelé « compte mail ») et de boîtes de messages.

Un compte de messagerie offre différents services.

1. Il permet d'authentifier l'utilisateur ; les informations nécessaires sont un nom et un mot de passe.
2. On peut relever le courrier d'un compte : chargement des messages depuis le serveur de messagerie vers le client.
3. On peut aussi envoyer un message : transfert du message à tous ses destinataires.

Pour effectuer la réception et l'émission des messages, le compte de messagerie s'appuie sur les services de 2 classes abstraites : **ServeurReception** et **ServeurEmission**.

1. Pour le **ServeurReception**, on précise la fréquence de relevé des messages (60 secondes par défaut) et si les messages relevés sont détruits ou non sur le serveur distant. Il possède une opération abstraite **chargerMessages** qui crée localement les messages récupérés.

2. Côté émission, on n'envisage ici qu'un type de serveur : `ServeurSMTP`. Celui-ci assure la transmission d'un message à un destinataire et fournit une indication permettant de savoir si l'envoi s'est bien passé ou non.

Représentez toutes ces informations sous forme d'un modèle de classes UML.

Localement les messages sont stockés dans des boîtes de messages. Il existe plusieurs types de boîtes de messages qui se différencient par leurs comportements. Il s'agit donc de bien identifier les services correspondants.

1. Pour chaque compte, il y a une boîte de réception, une boîte d'envoi, une corbeille.
2. L'utilisateur peut aussi créer autant de boîtes de messages personnelles qu'il le désire. On les désigne par le terme *boîte locale*.
3. Quand l'utilisateur rédige un nouveau message, il est placé dans la boîte d'envoi. Quand un message est reçu, il est stocké dans la boîte de réception.
4. On peut supprimer un message d'une boîte. Si cette boîte n'est pas la corbeille, le message est placé dans la corbeille. La corbeille peut être vidée en une seule opération.
5. On peut déplacer un message entre 2 boîtes locales, entre la boîte d'envoi et une boîte locale et entre la boîte de réception et une boîte locale.

Représentez toutes ces informations sous forme d'un modèle de classes UML.

4.3 Voisinage Cellule

Dans de nombreuses simulations, telles que les jeux vidéo, l'espace est découpé en *cellules*. Les comportements des personnages nécessitent de connaître les cellules vers lesquelles ils peuvent se déplacer ou celles dont ils peuvent percevoir le contenu. Ceci repose sur la notion de **voisinage**. Deux types de voisinage sont classiquement utilisés : le voisinage de MOORE (défini par les 4 cellules situées au nord, au sud, à l'ouest et à l'est) et le voisinage de VON NEUMANN (les 8 cellules adjacentes).

1. La classe `Cellule` implémente une opération de calcul des voisins qui retourne les cellules réellement atteignables. Pour cela, elle fait appel à un voisinage qui a pour responsabilité de retourner les cellules « voisines ». La cellule n'a pas à connaître le type d'objet qui rend

ce service ; la responsabilité de la cellule est de déterminer les cellules qui sont effectivement accessibles dans l'ensemble des cellules qui constituent son voisinage (il peut y avoir des cellules « interdites »).

2. La notion de voisinage se définit par le service `calculerVoisins` qui prend en entrée un nombre entier qui indique la « profondeur » du voisinage (les voisins directs ou les voisins des voisins etc) et qui retourne les cellules faisant parties du voisinage.
3. Deux façons de rendre ce service sont définies : le voisinage de MOORE (`VoisinageMoore`), le voisinage de VON NEUMANN (`VoisinageVN`).
4. Les classes dans lesquelles les algorithmes de calcul de voisinage sont implémentés ne sont modélisées dans cette étude.

Q.4.1 Représentez toutes ces informations sous forme d'un modèle de classes UML.

4.4 Album Photos

On s'intéresse ici aux fonctionnalités d'un logiciel, appelé `AlbumPhoto`, qui permet de gérer des photos numériques, de les importer depuis un appareil photo numérique, de les organiser, de leur appliquer des retouches...

Le logiciel gère deux types d'item : les albums et les photos. Un album peut contenir d'autres albums ou des photos. Ces deux types d'item sont identifiés par un nom. Ils ont aussi comme propriétés leur date de création et la date de leur dernière modification (on suppose que l'on dispose d'une classe `Date`). Il est possible de calculer la taille d'un item (l'information est fournie sous la forme d'un entier, le nombre d'octets). Dans le cas d'une photo, il s'agit de la taille de son fichier et dans le cas d'un album, de la somme des tailles des items qu'il contient.

Modélisez cette structuration des données manipulées par le logiciel ainsi que leurs propriétés sous forme d'un diagramme de classes UML.

Le logiciel permet d'appliquer différents types de retouche aux photos. On peut appliquer plusieurs retouches à une même photo. À tout moment, on peut savoir qu'elles retouches ont été appliquées à une photo et dans quel ordre.

Comme bon nombre de commande dans ce type de logiciel, l'effet obtenu suite à l'application d'une commande n'est pas toujours celui qu'on espérait ! Pour cela, il est possible d'annuler une

commande et de l'appliquer à nouveau. Les retouches implémentent ce mécanisme : elles peuvent être appliquées et annulées, services que peuvent offrir d'autres classes de l'application.

Un type particulier de retouche est l'application d'un filtre de couleur. Il consiste à changer les composantes R,G,B des pixels de la photo. Une autre retouche possible est de rogner la photo. Pour cela l'utilisateur définit un cadre (classe `Frame`) dont les propriétés sont les deux offsets en x et y des coins inférieurs gauche et supérieur droit. Appliquer cette retouche nécessite donc d'associer le cadre à la photo.

Modélisez ces propriétés qui supportent les fonctionnalités du logiciel sous forme d'un diagramme de classes UML.

5 Le modèle états-transitions : concepts de base

Sommaire

5.1	Objectif	94
5.2	Comportement de l'interface d'un téléphone	94
5.3	Utilisation d'un éditeur de document	95

5.1 Objectif

Ces exercices portent sur le modèle états-transitions du langage UML :

- ▷ états et des transitions ;
- ▷ déclencheurs des transitions : cas simples

5.2 Comportement de l'interface d'un téléphone

On considère le comportement simplifié de l'interface d'un téléphone mobile et on souhaite la modéliser en UML. On s'intéresse ici à la fonctionnalité 'appel d'un correspondant'.

Q 5.1. Quel modèle d'UML permet de représenter les fonctionnalités ?
Représentez la fonctionnalité 'appel correspondant' par un diagramme.

Description de la procédure d'appel.

1. Lorsqu'on active le téléphone, un écran d'accueil s'affiche.
2. Si l'on appuie sur la touche Fin, l'appareil se met en veille et aucune autre interaction n'est alors possible sans une réactivation de celui-ci.
3. Depuis l'écran d'accueil, la touche 'Composer' permet d'accéder à la fonctionnalité d'appel. Un écran d'invitation à composer un numéro s'affiche.
4. L'appui sur la touche 'Fin' ou sur la touche 'Accueil' ramène à l'écran d'accueil.

Q 5.2. Représentez ce comportement avec un diagramme états-transitions d'UML.

1. L'appui sur une touche numérique provoque l'affichage du chiffre correspondant à l'écran.
2. La touche 'Valider' provoque l'appel du numéro, sauf si aucun chiffre n'a été saisi.
3. Lors de l'appel, quand la ligne est détectée comme occupée, l'appareil affiche le message 'occupé' et, après 1 seconde, l'écran de composition s'affiche à nouveau.
4. Lors de l'appel, on peut abandonner l'opération en appuyant sur la touche 'Fin'.
5. Lors de l'appel, si le correspondant décroche, la communication est établie.
6. En cours de la conversation, si le correspondant raccroche, on affiche un message et on revient à l'écran d'accueil.
7. En cours de conversation, la touche 'Fin' interrompt la communication et l'écran d'accueil s'affiche à nouveau.

Q 5.3. Représentez ce comportement à l'aide d'un diagramme états-transitions UML.

Q 5.4. Identifiez-vous des ambiguïtés dans la description textuelle du comportement telle qu'elle vous est donnée ? Votre modélisation permet-elle de les lever ?

5.3 Utilisation d'un éditeur de document

L'étude porte sur une partie du comportement d'un éditeur de document tel qu'un traitement de texte, un éditeur graphique, un logiciel de retouche d'image...

1. Au lancement du logiciel, l'espace de travail est vide.
2. La combinaison de touches `ctrl-n` permet de créer un nouveau document et de l'éditer.
3. `ctrl-f` permet d'ouvrir un document existant dans un fichier : cela ouvre la fenêtre de sélection de fichier. Si l'utilisateur sélectionne un fichier, le document est ouvert et est éditabile. Si aucun fichier n'est sélectionné, on revient à l'état initial.

4. En mode "édition", `ctrl-s` permet d'enregistrer le document dans un fichier : s'il s'agit d'un nouveau document, l'utilisateur est invité à saisir un nom de fichier (fenêtre de saisie de fichier), si le document correspond à un fichier ouvert précédemment, `ctrl-s` l'enregistre dans ce même fichier. Dans les deux cas, on peut à nouveau éditer le fichier.
5. Pour enregistrer le document dans un nouveau fichier, la combinaison de touches est `shift-ctrl-s`.

Q 5.1. Quel type de machine à états est adapté à la modélisation de ce protocole opératoire ?

Q 5.2. Modélisez ce mode opératoire sous forme d'un diagramme états-transitions.

6 Le modèle états-transitions : concepts avancés

Sommaire

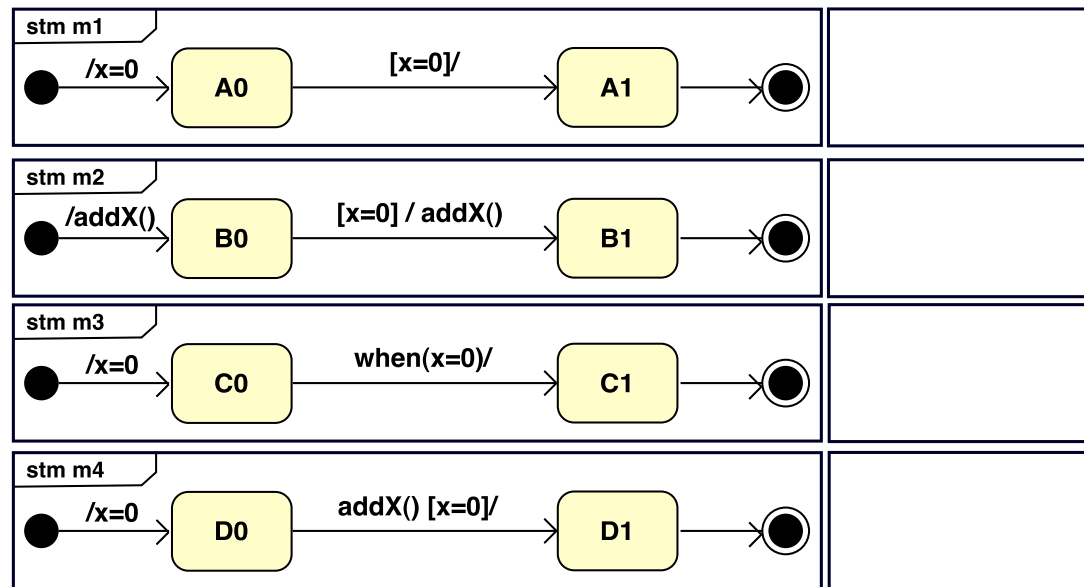
6.1	Objectif	97
6.2	Sémantique des machines à état	97
6.3	Comportement de l'interface d'un téléphone	100
6.4	Alternatives	101
6.5	Hiérarchie	102

6.1 Objectif

Ces exercices portent sur le modèle états-transitions du langage UML : modélisation comportementale; interprétation d'une machine à états; transitions et activités internes; états hiérarchiques (cas simples).

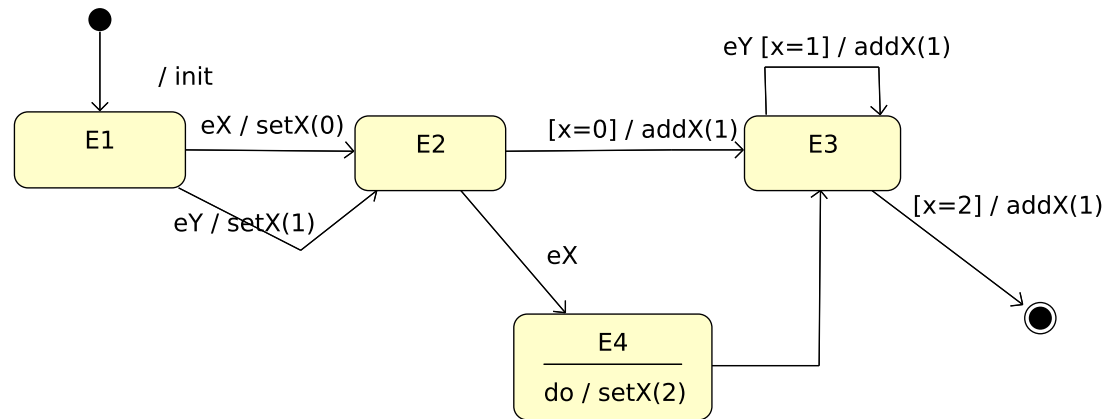
6.2 Sémantique des machines à état

On s'intéresse ici au comportement des objets d'une classe `CC` qui a un attribut `x: integer = 0` et deux opérations `setX(in v:integer)` et `addX(in v: integer = 1)`. La première affecte la valeur `v` à l'attribut `x`; la seconde incrémente l'attribut `x` de `v`.



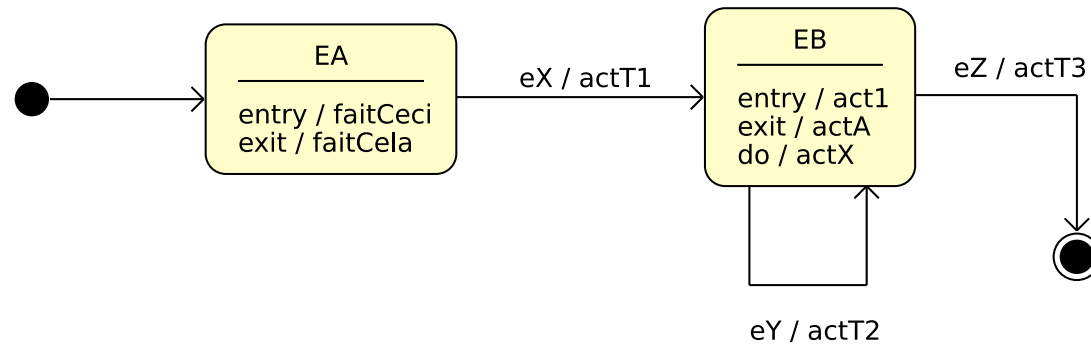
Q 6.1. Quelles sont les transitions franchissables ?

On associe maintenant la machine à états de la figure ci-dessous à la classe CC.



Q 6.2. Quelles transitions sont franchies si la séquence d'événements eX; eY; eX se produit ? Quelles sont les actions qui sont effectuées et dans quel ordre ? Même question avec la séquence eY; eX; eY.

On associe maintenant la machine à état de la figure ci-dessous à la classe CC.



Q 6.3. Quelles activités sont effectuées à l'activation de la machine à état ? Que se passe-t'il ensuite sur occurrence d'un événement eX ? Que se passe-t'il ensuite sur occurrence d'un événement eY ? Que se passe-t'il sur occurrence d'un événement eZ, l'état EB étant actif ?

6.3 Comportement de l'interface d'un téléphone

On reprend ici l'exercice sur la modélisation de l'interface d'un téléphone mobile sous forme d'un modèle états-transitions UML. On s'intéresse de nouveau à la fonctionnalité 'appel d'un correspondant'. L'objectif est maintenant d'utiliser des structures syntaxiques qui rendent le modèle plus compact, et donc plus lisible, et d'enrichir le modèle.

Q 6.1. Reprenez le modèle que vous avez déjà construit et regardez si vous ne pouvez pas en avoir une représentation plus simple en utilisant : des activités et transitions internes, des états hiérarchiques.

1. Appel d'un numéro du carnet d'adresses.

Dans cette version plus élaborée du téléphone, on peut aussi appeler un correspondant, non pas en tapant son numéro, mais en le sélectionnant dans le carnet d'adresses stocké dans le téléphone (on n'arrête pas le progrès!). On ne détaille pas ici la manière dont on réalise cette opération.

Q 6.2. Ajoutez cette nouvelle possibilité à votre modèle. Modifiez éventuellement le reste de votre modèle de façon à rendre ce genre d'ajout plus facile à l'avenir (on ne sait jamais avec le progrès...)

2. Enregistrement de l'appel dans le journal.

À la fin d'un appel, le numéro qui vient d'être composé est enregistré dans le journal des appels sortants (avec la date et l'heure).

Q 6.3. Ajoutez cette nouvelle fonctionnalité à votre modèle. (on vous avait prévenu au sujet du progrès...)

3. Gestion de l'éclairage lors de la composition d'un numéro.

Toutes ces petites merveilles de la technologie ont un gros défaut, elles consomment beaucoup d'énergie et les batteries ont une faible autonomie. En particulier, l'affichage est

gourmand en énergie. On a donc décidé de diminuer l'intensité de l'éclairage quand on n'a pas sollicité l'interface du téléphone depuis un certain temps. L'appui sur une touche provoque le retour à l'intensité normale.

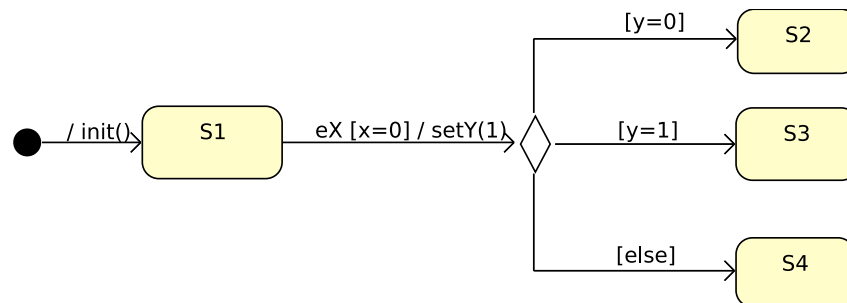
Q 6.4. Dans un premier temps, on n'implémente cet économiseur d'énergie uniquement lors de la composition du numéro. Modifiez votre modèle en conséquence.

Q 6.5. Maintenant, on généralise ce mode à l'ensemble des fonctionnalités du téléphone. Modifiez votre modèle en conséquence.

6.4 Alternatives

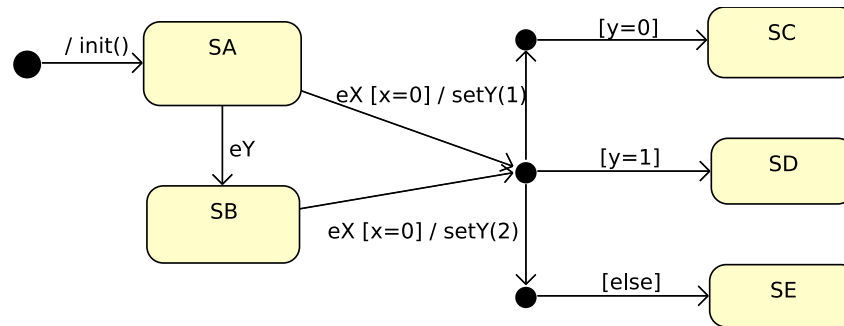
On considère le comportement suivant des objets d'une classe CA qui a un attribut `x`: `integer = 0` et `y`: `real` et deux opérations `init()` et `setY(in v: integer)`. La première affecte la valeur 0 aux attributs `x` et `y`; la seconde affecte la valeur `v` à l'attribut `y`.

On associe la machine à état de la figure ci-dessous à la classe CA.



Q 6.1. Quel est l'état d'une instance de CA suite à l'occurrence d'un événement de type `eX` ?

On associe maintenant la machine à état de la figure ci-dessous à la classe CA.



Q 6.2. Quel est l'état d'une instance de CA suite à l'occurrence d'un événement de type `eX` ?
Même question pour un événement `eY` suivi de `eX`.

6.5 Hiérarchie

On considère ici la gestion des états des fenêtres d'une application, telle que l'on connaît sur différents systèmes d'exploitation.

1. Les états fondamentaux d'une fenêtre sont : normale, réduite et agrandie (plein écran).
2. À la création de la fenêtre, celle-ci est ouverte, avec une taille normale.
3. Si la fenêtre n'est pas en plein écran, on peut la déplacer et changer ses dimensions.
4. Si elle est en plein écran et que l'on change la résolution de l'affichage, il faut la redimensionner.
5. Quand la fenêtre est ouverte, on peut l'agrandir, puis revenir à sa taille normale.
6. Étant ouverte, on peut la réduire ; si on l'ouvre à nouveau, la fenêtre reprend sa taille.

Q 6.1. Modélisez ce comportement sous forme d'un diagramme états-transitions UML. Vérifiez que, sous le système d'exploitation que vous utilisez, ce comportement est bien respecté. Corrigez éventuellement votre modèle.

7 Le modèle des activités : processus métier

Sommaire

7.1	Objectif	103
7.2	Modélisation d'un processus métier	103

7.1 Objectif

Ce travail porte sur le modèle des activités du langage UML :

- ▷ séquencement et alternatives ;
- ▷ flots de contrôle et de données ;
- ▷ partition des activités.

7.2 Modélisation d'un processus métier

L'objectif est de modéliser un processus métier sous la forme d'un diagramme d'activités UML. L'étude porte sur le processus de développement d'un logiciel, plus particulièrement les phases suivantes : conception, réalisation (codage) et tests unitaires. Il ne s'agit que d'un cas d'école qui s'inspire librement de la réalité.

Première approche : la ligne droite.

Il s'agit ici de commencer par le cas le plus simple où la réalisation de chaque activité est un succès ; c'est le scénario sans échec, appelé parfois scénario nominal. Dans la suite, on envisagera les alternatives à ce scénario idéal.

1. La première étape est celle de l'édition des modèles ; elle se fait en phase de conception.
2. Quand la modélisation est terminée, on génère le code.
3. Le processus se poursuit par la production des exécutables et la mise au point du code (déroulement pas à pas du code au debugger, détection des éventuels problèmes d'allocation...). Ces activités s'inscrivent dans la phase de codage.

4. La dernière étape du processus, tel qu'il est envisagé ici, est celle de passation des tests unitaires, parfois réalisée par une équipe spécialisée.

Q.7.1 : Représentez cette première version du processus sous forme d'un diagramme d'activités UML.

Q.7.2 : Faites en sorte que l'organisation des activités soit bien visible en les partitionnant par phase.

Un processus plus itératif.

S'imaginer que l'on peut produire un modèle complet et sans erreur permettant une génération complète du code en une seule passe est un leurre. Dans la pratique, il est d'ailleurs préférable de mener la phase de conception en réalisant de courtes itérations sur les activités de modélisation et de codage¹.

1. L'activité de génération de code à partir du modèle se fait tant que la modélisation est considérée comme incomplète. Si le modèle est complet, alors on passe aux tests unitaires.
2. Le générateur de code effectue des vérifications sur la structure du modèle. Il peut, par exemple, détecter que le type d'un attribut n'a pas été spécifié... Si le modèle est incomplet, le générateur ne produit rien et il faut reprendre la modélisation ; sinon, on peut passer à la phase de codage.

Q.7.3 : Reprenez votre diagramme d'activités en introduisant ces alternatives. Utilisez pour cela les nœuds de décision et de fusion.

Procédures de test.

L'activité de test est délicate et coûteuse en temps. Pour la mener à bien, il faut rédiger au préalable des « spécifications de test » qui décrivent les programmes de test, les procédures de réalisation des tests, les données d'entrée et les résultats attendus.

1. À condition que les choix d'architecture soient au préalable validés, ce qui est hors du champ de cette modélisation.

1. La réalisation des tests suppose que l'on ait à sa disposition un exécutable et la spécification des tests.
2. Une fois les tests réalisés, si un écart est constaté entre les résultats attendus et ceux du test, alors on rédige un rapport d'anomalie. Ce rapport est une donnée d'entrée pour la conception du logiciel.

Q.7.4 : Reprenez votre diagramme d'activités en faisant figurer ces flots d'activités. Veillez à la cohérence des flots d'exécution.

8 Le modèle des activités : du modèle au programme

Sommaire

8.1	Objectif	106
8.2	Rétro-ingénierie : du code au modèle	106
8.3	Traduction d'un modèle en langage de programmation	108

8.1 Objectif

Ce travail porte sur l'utilisation du modèle des activités du langage UML en phase de conception et de codage d'un logiciel. Il s'agit de voir comment on peut traduire un tel modèle en langage de programmation objet, ici en java.

Le cas d'étude est celui d'une version très simplifiée d'un simulateur de la vie d'animaux dans leur environnement : des lions, des gnous, des gazelles dans une savane.

8.2 Rétro-ingénierie : du code au modèle

Pour faire vivre chaque animal peuplant la savane, le simulateur, à chacun de ses cycles, appelle la méthode `live()` de la sous-classe d'`Animal`. Cette méthode correspond donc à l'exécution du comportement de l'animal à chaque pas de temps. Du manière générale, les animaux naissent, se déplacent, se nourrissent, se reproduisent dans la savane, et ils finissent par mourrir. La savane est une étendue plane qui est discrétisée en cellules, `Cell2D`, pour les besoins de la simulation. Ces cellules peuvent être de différents types : couvertes d'herbe, `Grass` ou d'arbres, `Tree`. Le diagramme de la figure 1 présente les principales propriétés des classes sur lesquelles reposent les fonctionnalités du simulateur.

L'objectif ici est d'analyser le code de la méthode `live()` de la classe `Lion` et de le transcrire en un modèle d'activités. Vous trouverez le code de cette méthode dans l'archive disponible sur Moodle. Ce code est le résultat de la transformation du modèle de classes qui vous est fourni et qui a été produit en utilisant le logiciel de modélisation `bouml`.

Q.8.1 : Représentez la méthode `live` de la classe `Lion` sous forme d'un diagramme d'activités UML.

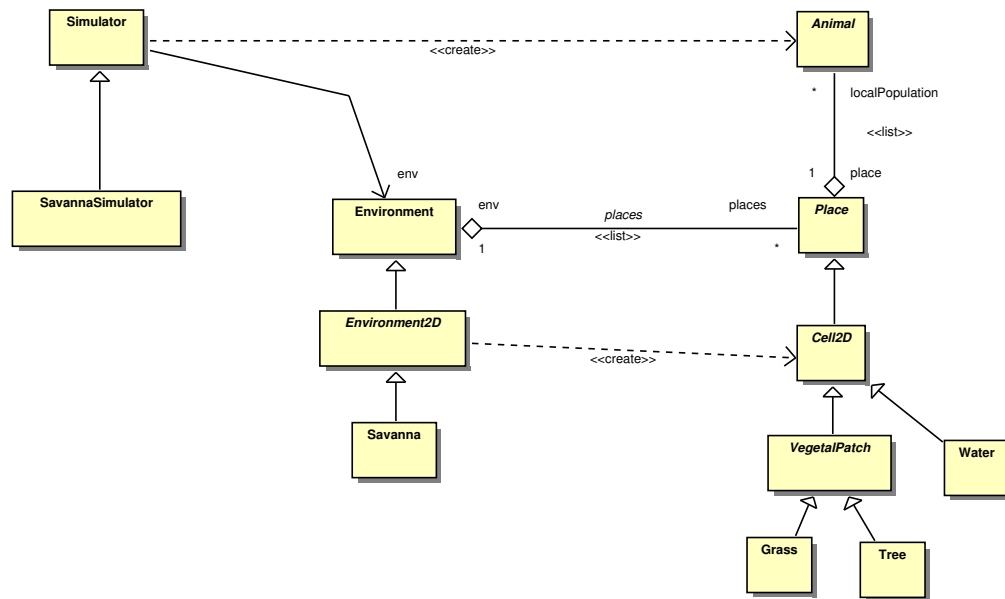


FIGURE 1 – Modèle de classes du simulateur de la vie des animaux dans la savane

8.3 Traduction d'un modèle en langage de programmation

On s'intéresse maintenant à un des comportements des lions qui est leur mode de chasse. Ce comportement est décrit sous forme d'un diagramme d'activité sur la figure 2.

La figure 3 donne l'ensemble des propriétés de la classe `Lion`. En cas de besoins, vous pouvez consulter le modèle sous bouml.

Q.8.1 : Écrivez le code de l'opération `hunt` de la classe `Lion` conformément au diagramme d'activités de la figure 2.

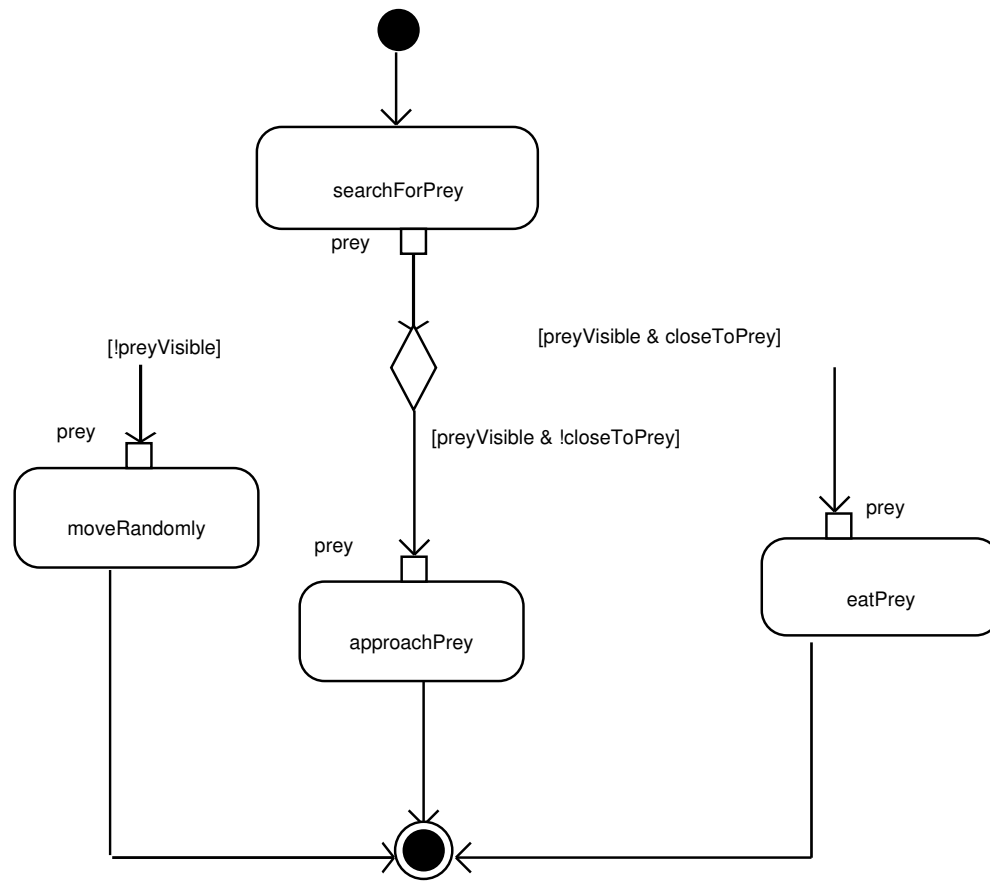


FIGURE 2 – Modèle d'activités de l'opération `hunt` de la classe `Lion`

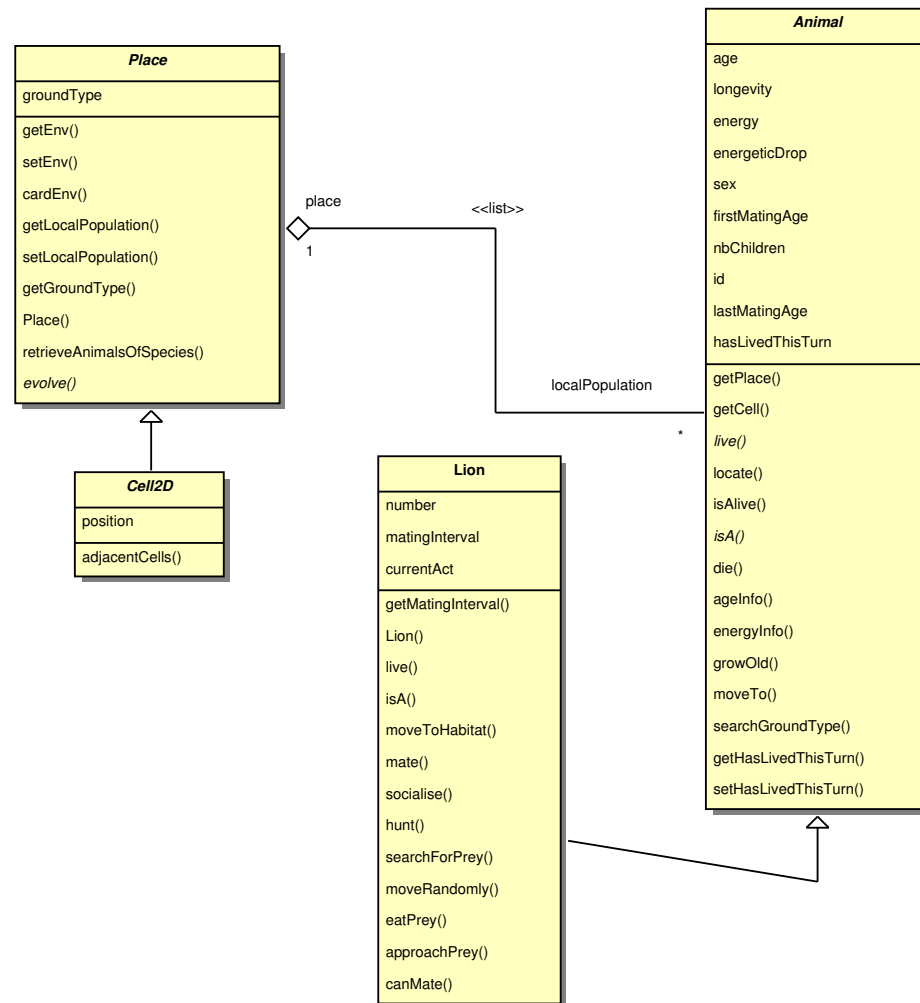


FIGURE 3 – La classe Lion et ses dépendances

Solution Labo

9 Le modèle états-transitions

Sommaire

9.1	Concepts de base	111
9.1.1	Exo Interface d'un téléphone	111
9.1.2	Exo Éditeur de document	114
9.2	Concepts avancés	115
9.2.1	Exo Sémantique des machines à état	115
9.2.2	Exo Hiérarchie	116

9.1 Concepts de base

9.1.1 Exo Interface d'un téléphone

Figure 9.1. *Fonctionnalités*

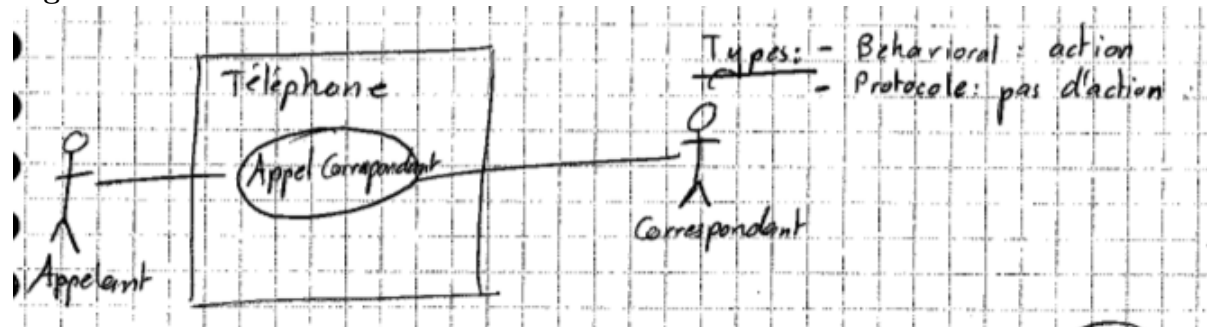


Figure 9.2. Diagramme ET

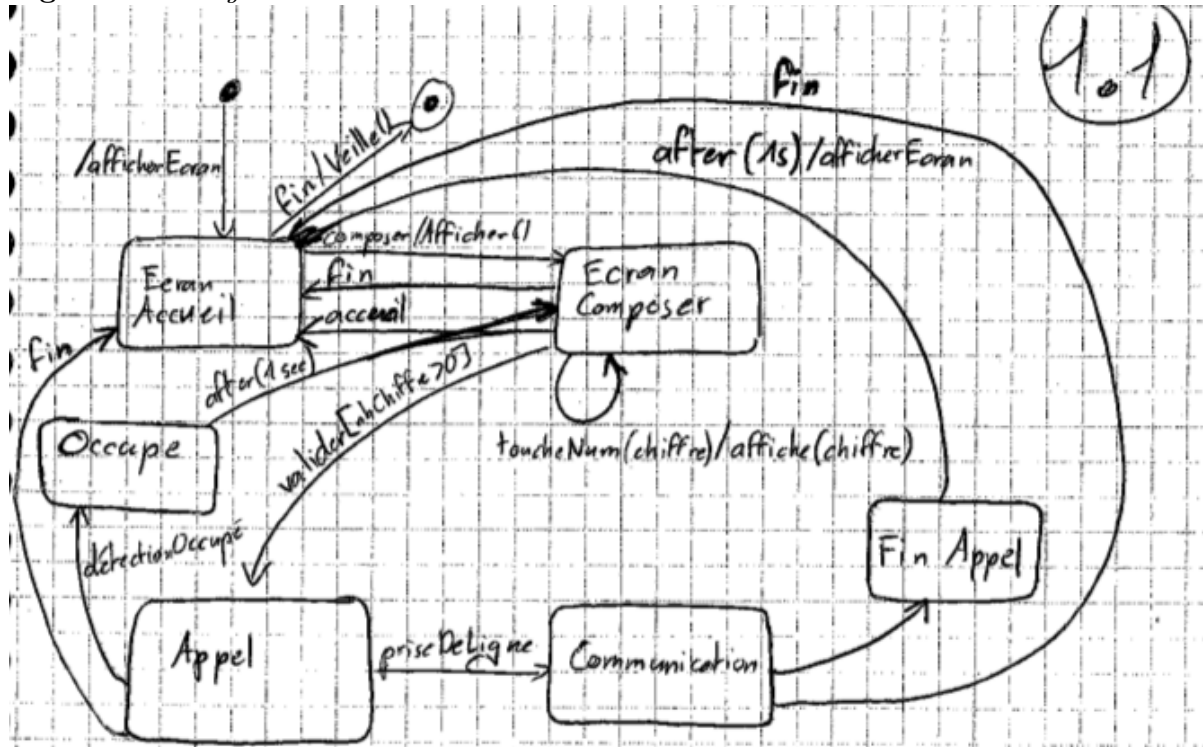
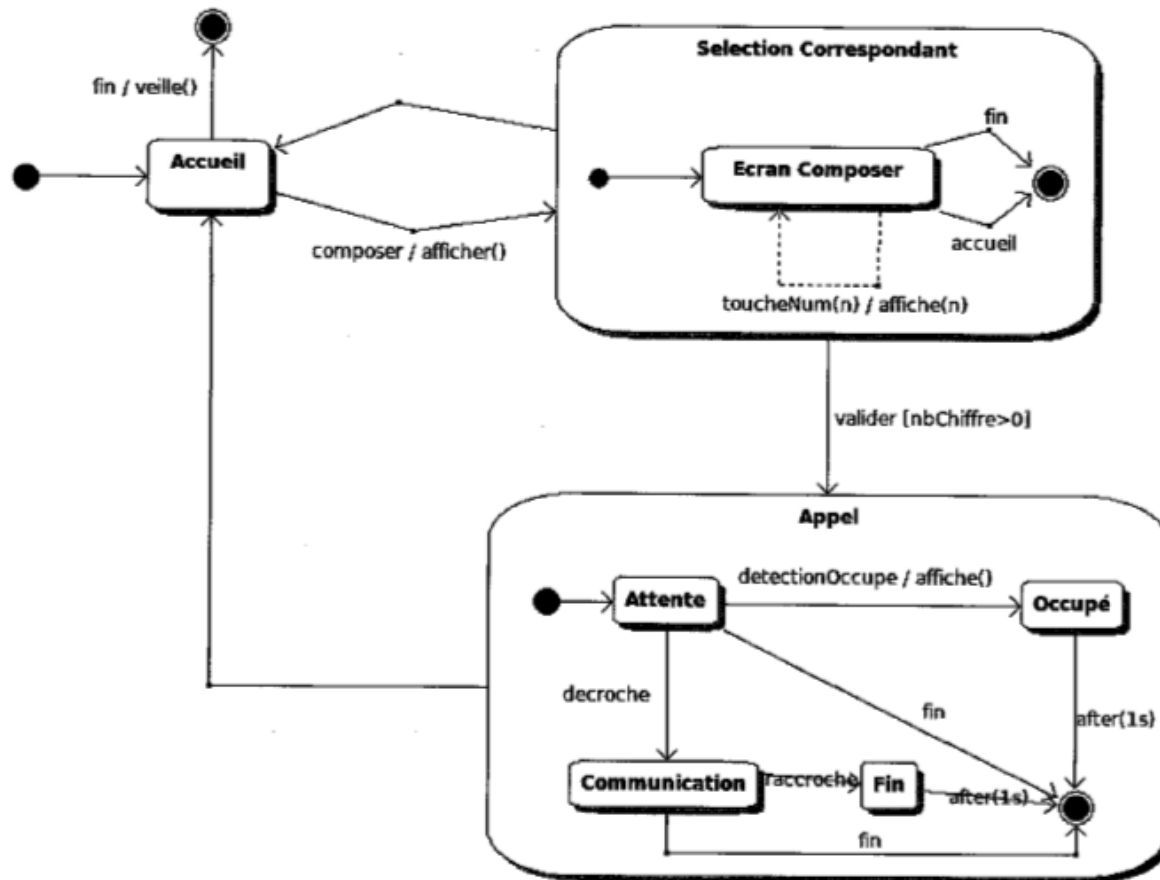
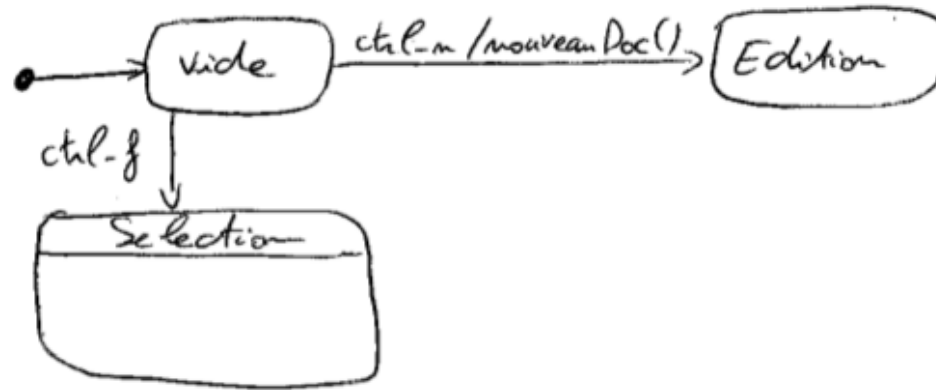


Figure 9.3. *Diagramme ET 2*



9.1.2 Exo Éditeur de document

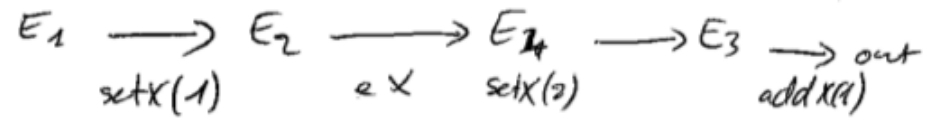
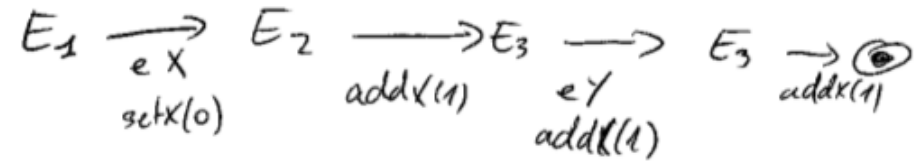
Figure 9.4. *Diagramme ET*



9.2 Concepts avancés

9.2.1 Exo Sémantique des machines à état

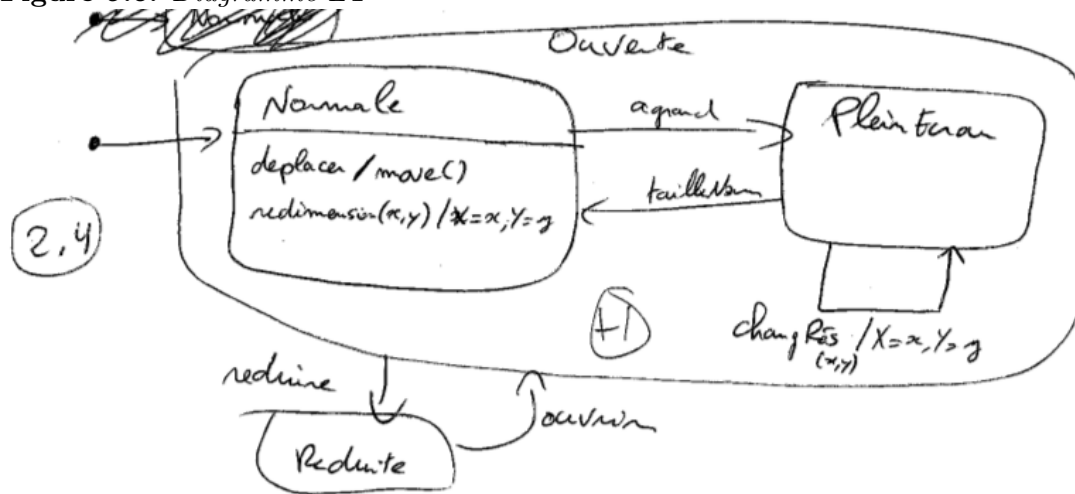
Figure 9.5. Transitions franchissables



- 1) fait ça
- 2) fait ça, actT1, act1, actX
- 3) actA, actT2, act1, actX
- 4) actA, actT3

9.2.2 Exo Hiérarchie

Figure 9.6. Diagramme ET



10 Le modèle d'activités

Sommaire

10.1 Processus métier	117
10.1.1 Exo Modélisation d'un processus m'étier	117

10.1 Processus métier

10.1.1 Exo Modélisation d'un processus m'étier

Figure 10.1. *Première approche*

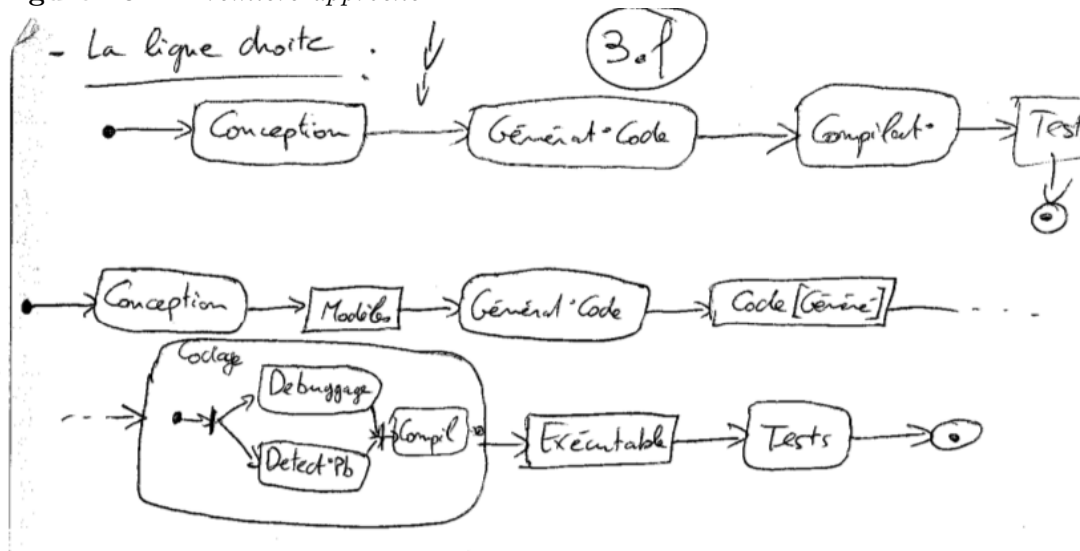


Figure 10.2. Première approche 2

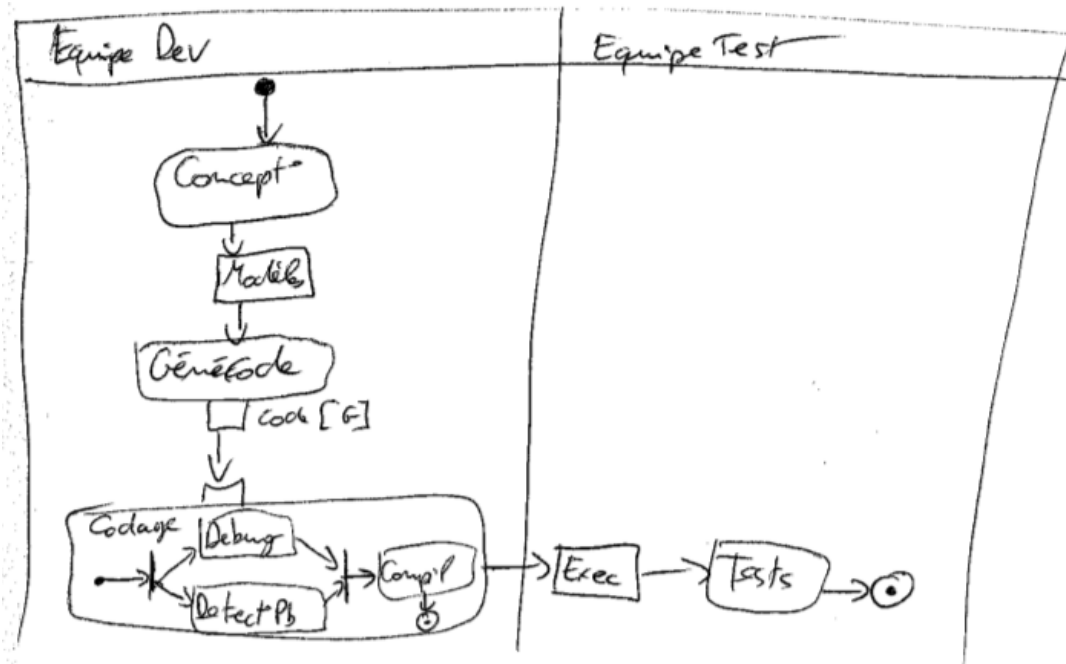


Figure 10.3. *Processus plus iteratif*

